

Partially Non-Interactive Two-Round Threshold and Multi-Signatures with Tighter and Adaptive Security

Yanbo Chen*

February 23, 2026

Abstract

Threshold and multi-signature schemes enable a group of signers to jointly sign messages. In the pairing-free discrete-logarithm setting, there have been extensive research efforts on two-round threshold and multi-signature schemes. A prominent category is partially non-interactive schemes, where the first round is message-independent and can be preprocessed offline. However, there exist gaps in security properties between partially non-interactive schemes and fully online schemes. While there are fully online schemes that achieve rewinding-free or fully adaptive security based on standard assumptions, existing partially non-interactive schemes require the adversary to act algebraically or rely on non-standard assumptions for these properties.

We bridge this gap by proposing a family of new constructions of partially non-interactive threshold and multi-signature schemes. Central to our constructions is a technique that renders HBMS (Bellare and Dai, Asiacrypt 2021) and its variants partially non-interactive. Specifically, we present the following instances:

- The first partially non-interactive two-round multi-signature scheme with a rewinding-free reduction under standard assumptions. This scheme provides a tighter security guarantee against non-algebraic adversaries than all prior partially non-interactive schemes.
- The first partially non-interactive two-round threshold signature scheme achieving fully adaptive security against non-algebraic adversaries under standard assumptions.

1 Introduction

Threshold signature schemes [Des88] and multi-signature schemes [IN83] are protocols for distributed signature generation. Threshold signatures distribute the signing key among a group of users, such that any subgroup of users that exceeds a threshold can sign on behalf of the group. Multi-signatures allow users with individual key pairs to dynamically form a group and jointly sign. Threshold and multi-signatures have both received renewed attention for applications in cryptocurrencies (e.g., [GGN16, MPSW19]).

This work focuses on threshold and multi-signature schemes over pairing-free groups. Unlike pairing-based signatures [BLS01] which typically support efficient, non-interactive distributed signing [Bol03], practical pairing-free threshold and multi-signatures require the signers to interact. Naturally, the number of rounds becomes a central criterion, and a number of schemes have realized two-round signing protocols.

Among the two-round schemes, a family has the property that their first round can be preprocessed without knowing the message and the signer set. Therefore, with preprocessing, the on-line signing phase after the message and signer set are determined is effectively non-interactive.

*University of Ottawa. Email: ychen918@uottawa.ca.

Such schemes enjoy some advantages of non-interactive protocols and are called *partially non-interactive*. This family consists of the threshold Schnorr scheme FROST [KG20], Schnorr multi-signature schemes DWMS/MuSig2 [AB21, NRS21], and their variants [KG20, BCK+22, RRJ+22, HDL+25, BCL+25, BGC+25].

On the other hand, there are a number of interesting two-round schemes where both rounds are online, yet they achieve more favorable security results:

- Tighter security: [PW23, PW24, TSS+24, BW25, LÖ25] proposed two-round multi-signature schemes with rewinding-free security reductions in the ROM. Such reductions avoid invoking the forking lemma [PS96, BN06] and hence enjoy tighter or even fully tight security bounds. There also exists a two-round threshold signature scheme [Che25a] with rewinding-free tighter security. In comparison, the security proofs for existing partially non-interactive threshold and multi-signature schemes rely on either the algebraic group model (AGM) [FKL18] or rewinding. The AGM requires the adversary to act algebraically, while rewinding results in a quadratic security loss. Multi-signatures with public key aggregation [NRS21, TZ23] even require nested rewinding and thus suffer from a quartic loss.
- Adaptive security: The adaptive security model considers an adversary that can adaptively corrupt users at any time to obtain their secret key shares and full history. There have been fully online two-round threshold signatures with full adaptive security in the ROM from standard assumptions [Che25a, Che25b, GCRS25].

By contrast, FROST and its variants [KG20, CKK+25, BCL+25, BGC+25] are only proved fully adaptively secure in the AGM in addition to the ROM. Indeed, several works provide meta-reductions [CKM25, BCL+25] that formally establish barriers to proving their adaptive security without the AGM.

1.1 Contributions

A Family of New Partially Non-Interactive Schemes.

We propose a family of new constructions of partially non-interactive two-round threshold and multi-signatures. The key to our constructions is an elegant modification to the two-round scheme HBMS by Bellare and Dai [BD21] that makes the first-round preprocessable with only minor overhead. Particularly, our modification does not increase the signature size. All our schemes produce signatures consisting of three field/group elements.

While HBMS is based on DL, the scheme TSSHO [TSS+24] can be viewed as a DDH-based variant of HBMS. Moreover, HBMS has a straightforward threshold signature version called HBTS [Che25b]. Therefore, our modification on HBMS provides a family of threshold and multi-signature schemes offering both DL-based and DDH-based instantiations. We call them HBMS2[DL], HBMS2[DDH], HBTS2[DL], HBTS2[DDH]. We can also apply the masking technique from [KRT24, Che25b] to our threshold signature schemes to obtain adaptively secure schemes HBTS2[DL]-Mask, HBTS2[DDH]-Mask. Among these schemes, we present HBMS2[DDH] and HBTS2[DDH]-Mask as our primary contributions, which achieve the desirable security results that we mentioned and previously only achieved by schemes with two online rounds.

Multi-Signatures with Tighter Security.

Our multi-signature scheme HBMS2[DDH] supports non-interactive public-key aggregation. Like TSSHO [TSS+24], our scheme can be proved secure in the ROM by a rewinding-free security reduction. Consequently, our scheme has a security loss of $\Theta(q_h)$, where q_h is the number of hash queries made by the adversary. Namely, the security reduction that solves DDH succeeds with probability $\Theta(\varepsilon/q_h)$, where ε is the success probability of any adversary that breaks the security of HBMS2[DDH]. In comparison, existing partially non-interactive schemes with key aggregation all rely on a nested

Scheme	Round	Sig.	PK.	Comm.	Assumption	Bound
Musig-DN [NRSW20]	2	$1 \mathbb{G} + 1 \mathbb{Z}_p $	$1 \mathbb{G} $	$2 \mathbb{G} + 1 \mathbb{Z}_p + \pi $	DDH	ε^4/q_h^3
HBMS [BD21]	2	$1 \mathbb{G} + 2 \mathbb{Z}_p $	$1 \mathbb{G} $	$1 \mathbb{G} + 2 \mathbb{Z}_p $	DL	$\varepsilon^4/(q_s^4 q_h^3)$
Chopsticks I [PW23]	2	$3 \mathbb{G} + 4 \mathbb{Z}_p $	$2 \mathbb{G} $	$3 \mathbb{G} + 1 \mathbb{Z}_p + \lambda$	DDH	ε/q_s
Toothpicks I [PW24]	2	$3 \mathbb{Z}_p + 2\lambda$	$2 \mathbb{G} $	$2 \mathbb{G} + 1 \mathbb{Z}_p + \lambda$	DDH	ε/q_s
TSSHO [TSS+24]	2	$3 \mathbb{Z}_p $	$2 \mathbb{G} $	$2 \mathbb{G} + 2 \mathbb{Z}_p $	DDH	ε/q_s
T-Spoon [BW25]	2	$2 \mathbb{G} + 9 \mathbb{Z}_p $	$2 \mathbb{G} $	$3 \mathbb{G} + 1 \mathbb{Z}_p + \lambda + 1$	DDH	ε
MuSig2 [NRS21]	1+1	$1 \mathbb{G} + 1 \mathbb{Z}_p $	$1 \mathbb{G} $	$4 \mathbb{G} + 1 \mathbb{Z}_p $	AOMDL	ε^4/q_h^3
TZ [TZ23]	1+1	$1 \mathbb{G} + 2 \mathbb{Z}_p $	$1 \mathbb{G} $	$4 \mathbb{G} + 2 \mathbb{Z}_p $	DL	ε^4/q_h^3
HBMS2[DDH]	1+1	$3 \mathbb{Z}_p $	$2 \mathbb{G} $	$4 \mathbb{G} + 2 \mathbb{Z}_p $	DDH	ε/q_h

Table 1: Comparison between pairing-free two-round multi-signature schemes with key aggregation. We compare the round complexity of the signing protocol, the signature size, the public-key size, the per-signer communication complexity, the underlying hardness assumption, and the security bound. For round complexity, “1+1” means that the first round is offline preprocessable. We let $|\mathbb{G}|$ and $|\mathbb{Z}_p|$ denote the sizes of a group element and a field element, respectively. We use λ to denote the security parameter. For Musig-DN, $|\pi|$ denotes the size of a zk-SNARK that proves correct computation of deterministic nonces. The security bound is the success probability of the reduction that solves the underlying hard problem, expressed as a function of the adversary’s success probability ε , ignoring Θ factors. For the security bound, we only consider proofs in the ROM without the AGM.

rewinding security reduction. Their security bounds are $\Theta(\varepsilon^4/q_h^3)$. Even without key aggregation, they still rely on rewinding and have a bound of $\Theta(\varepsilon^2/q_h)$ that suffers from a quadratic loss.

We provide some concrete numbers to give some sense about these bounds. Consider standard cryptographic groups of order p with $p \approx 2^{256}$. Assume a τ -time algorithm can solve the DL-related hard problems with probability at most τ^2/p (ignoring Θ factors). Consider an adversary with $\tau \approx q_h \approx 2^{60}$. Then our bound guarantees a success probability of about 2^{-76} . The bound $\Theta(\varepsilon^2/q_h)$ for previous schemes without key aggregation only guarantees 2^{-38} , while the bound $\Theta(\varepsilon^4/q_h^3)$ for previous schemes with key aggregation offers no meaningful guarantee.

We remark that our HBMS2[DDH] is not fully tight. Furthermore, our scheme has looser security than its fully online counterpart [TSS+24]. Our scheme has a security loss of $\Theta(q_h)$, while [TSS+24] achieves a security loss of $\Theta(q_s)$, where the number of hash queries q_h is typically considered larger than the number of signing queries q_s . We consider further improving the tightness for partially non-interactive schemes an interesting future direction.

Table 1 compares our HBMS2[DDH] with existing pairing-free two-round multi-signature schemes with key aggregation.¹

Threshold Signatures with Adaptive (and Tighter) Security.

Our DDH-based threshold signature scheme HBTS2[DDH] also achieves a tighter reduction for static security than existing partially non-interactive schemes in the ROM. Moreover, when combining with the masking technique [KRT24, Che25b], we obtain HBTS2[DDH]-Mask with full adaptive security in the ROM. This is the *first* partially non-interactive threshold signature scheme proved to achieve full adaptive security in the ROM from standard assumptions.

Previously, only FROST [KG20, CKK+25] and ms-FROST [BCL+25] are proved fully adaptively secure, based on the algebraic one-more discrete logarithm (AOMDL) assumption, but both in the AGM. In addition, FROST *necessarily* requires a new hardness assumption unrelated to DL called Low-Dimensional Vector Representation (LDVR) [CS25, CKK+25]. Our HBTS2[DDH]-Mask avoids both AGM and LDVR.

The masking technique by [KRT24] is somewhat generic and can be applied to several different

¹We did not list [AB21, HDL+25, LÖ25]. DWMS [AB21] is highly similar to MuSig2 but is only proved secure in the AGM. [HDL+25] is a privacy-preserving variant of MuSig2. Skewer-PF [LÖ25] is a fully online two-round scheme that achieves tight multi-user security, yet it requires proofs of possession.

Scheme	Round	Sig.	PK.	Comm.	Assumption	AGM?	Bound
Dazzle [Che25a]	2	$1 \mathbb{G} + 3 \mathbb{Z}_p $	$2 \mathbb{G} $	$4 \mathbb{G} + 2 \mathbb{Z}_p $	DDH	✗	ε/q_s
HBTS-Mask [Che25b]	2	$1 \mathbb{G} + 2 \mathbb{Z}_p $	$1 \mathbb{G} $	$1 \mathbb{G} + 2 \mathbb{Z}_p $	DL	✗	$\varepsilon^2/(q_s^2 q_h)$
GCRS [GCRS25]*	2	$1 \mathbb{G} + 1 \mathbb{Z}_p $	$1 \mathbb{G} $	$\Theta(\log d)(\mathbb{G} + \mathbb{Z}_p)$	DDH	✗	ε^2/q_h
FROST [KG20, CKK+25]	1+1	$1 \mathbb{G} + 1 \mathbb{Z}_p $	$1 \mathbb{G} $	$2 \mathbb{G} + 1 \mathbb{Z}_p $	AOMDL, LDVR	✓	ε
ms-FROST [BCL+25, BGC+25]	1+1	$1 \mathbb{G} + 1 \mathbb{Z}_p $	$1 \mathbb{G} $	$2 \mathbb{G} + 1 \mathbb{Z}_p $	AOMDL	✓	ε
HBTS2[DDH]-Mask	1+1	$3 \mathbb{Z}_p $	$2 \mathbb{G} $	$4 \mathbb{G} + 2 \mathbb{Z}_p $	DDH	✗	ε/q_h

Table 2: Comparison between pairing-free two-round threshold signature schemes with full adaptive security. We compare the round complexity of the signing protocol, the signature size, the public-key size, the per-signer communication complexity, the underlying hardness assumption, whether the proof of adaptive security relies on the AGM, and the security bound. The security bound is the success probability of the reduction that solves the underlying hard problem, expressed as a function of the adversary’s success probability ε , ignoring Θ factors.

*: After issuing a pre-specified bound of d signatures, GCRS requires the signers to refresh their keys.

schemes to gain adaptive security. Chen [Che25b] applies it to FROST to achieve non-AGM full adaptive security, but that results in a three-round or stateful two-round scheme. ms-FROST [BCL+25] is another way to apply the masking technique to FROST, which yields a partially non-interactive two-round scheme but requires the AGM.

As observed by [Che25b], HBTS is more masking-friendly, in the sense that it can be masked to achieve non-AGM full adaptive security without increasing the round number or making the scheme stateful. This turns out to be also true for our HBTS2. We remark that being rewinding-free is not necessary for the masking technique to be applicable. Masking our HBTS2[DL] would give a fully adaptively secure scheme HBTS2[DL]-Mask based on DL (but with a looser reduction).

Table 2 compares our HBTS2[DDH]-Mask with existing pairing-free two-round threshold signature schemes with full adaptive security.

1.2 Related Work

Multi-Signatures and Tighter Security.

Provably secure multi-signatures in early literature rely on interactive key generation protocols [MOR01] or the knowledge of secret key (KOSK) assumption [Bol03, LOS+06]. Bellare and Neven proposed the first multi-signature scheme in the *plain public-key model* [BN06], where each signer locally generates its own key and is not assumed to have proved the knowledge of its secret key. This model is now widely accepted as the most robust and realistic one for multi-signatures. We remark that there is a substantial body of work considering the Proof of Possession (PoP) model [RY07], similar to yet weaker than KOSK.

Recent years have seen a resurgence of interest in multi-signatures, largely driven by their application in cryptocurrency as highlighted in [MPSW19]. The work also formalized *public-key aggregation*, which allows a group of signers to only publish a compact aggregated key for verification. Support for public-key aggregation has become a common practice in designing multi-signature schemes.

In the pairing-free discrete logarithm setting, early three-round schemes were proposed in [MOR01, BN06, BJ08, BDN18, MPSW19]. While some early two-round proposals [BCJ08, MWLD10, STV+16, MPSW19] were shown to be insecure [DEF+19, BLL+21], a number of provably secure two-round schemes later emerged [DEF+19, NRSW20, AB21, NRS21, BD21, TZ23, PW23, PW24, RSY24, TSS+24, HDL+25, BW25, LÖ25], where [AB21, NRS21, TZ23, HDL+25] are partially non-interactive. [BDPT25] also proposed an ordered multi-signature scheme.

However, a major challenge in this domain, as well as in threshold signatures, remains the tightness of security reductions. The security of cryptographic schemes is usually proved by building a reduction that given access to an adversary breaking the security with probability ε_a , breaks the underlying hard problem with probability ε_r . All the previously known partially non-interactive two-round threshold and multi-signature schemes over pairing-free groups are proved secure in the AGM or with rewinding reductions. Such reductions establish a loose bound $\varepsilon_r \geq \varepsilon_a^2/\Theta(q_h)$. For multi-signature schemes with public-key aggregation, the reductions apply nested rewinding and obtain an even looser bound $\varepsilon_r \geq \varepsilon_a^4/\Theta(q_h^3)$.

Such loose security bounds constitute a significant drawback for concrete security and parameter choices. To address this, a line of research has successfully constructed multi-signatures [FH21, PW23, PW24, BW25, LÖ25] and threshold signatures [Che25a, BW26] with fully tight security. Other rewinding-free yet non-tight schemes were proposed in [PW23, PW24, TSS⁺24]. Furthermore, tightness of security proofs has also been studied in [BD21, RSY24, SS24]. Despite these advances, none of the threshold or multi-signature schemes achieving tighter bounds or rewinding-free security are partially non-interactive.

Threshold Signatures and Adaptive Security.

Initial work in pairing-free discrete logarithm-based threshold signatures [GJKR07, CGJ⁺99, SS01] focused on robust schemes. Since the proposal of FROST [KG20], which compromises on robustness for a lightweight protocol, many works considered threshold signatures in fewer rounds [BCK⁺22, RRJ⁺22, TZ23, CGRS23, Lin24].

A critical aspect of these schemes is their security model. A basic model is the *static* model, where the adversary first statically corrupts t users before key generation. A stronger, more realistic model is the *adaptive* model, allowing corruption after initiation. Adaptive security was considered in early robust schemes [CGJ⁺99, FMY99, JL00, LP01, AF04], and its study for lightweight schemes was initiated by [BL22, Mak22, CKM23]. A generic guessing argument reduces adaptive security to static security at the cost of a factor of $\binom{n}{t}$. While equivalent for small groups, this loss grows exponentially with n . For instance, for $n = 128$ and $t = 42 \approx 128/3$, the loss exceeds 2^{113} , meaning a scheme with 128 bits of static security essentially provides no confidence against adaptive attacks.

This discrepancy highlights the need for natively adaptively secure schemes. FROST is a state-of-the-art threshold Schnorr scheme with a partially non-interactive two-round signing protocol. However, recent progress [CS25, CKK⁺25] showed that its full adaptive security necessarily relies on an additional, unusual assumption called Low-Dimensional Vector Representation (LDVR), unrelated to DL. Moreover, even assuming LDVR, its full adaptive security is proved in the AGM [CKK⁺25], with a meta-reduction [CKM25] establishing a barrier to a non-AGM proof.

There has been a subsequent line of work constructing adaptively secure threshold signature schemes from standard, well-understood assumptions [BLT⁺24, KRT24, BDLR25c, Che25a, BDLR25b, Che25b, GCRS25, NRT25, BCL⁺25, BGC⁺25], mostly without the AGM. Among these, [Che25a, Che25b, GCRS25, BCL⁺25, BGC⁺25] proposed fully adaptively secure two-round schemes. Unfortunately, the only partially non-interactive scheme *ms*-FROST, independently proposed in [BCL⁺25] and [BGC⁺25], is indeed the only one relying on the AGM. Similar to FROST, a barrier to a non-AGM proof for it is established by a meta-reduction [BCL⁺25]. Prior to our work, there exists no partially non-interactive scheme achieving non-AGM fully adaptive security.

Finally, we mention that other important research directions in the pairing-free discrete-logarithm setting include threshold ECDSA [GGN16] and robustness in asynchronous networks [RRJ⁺22]. Such schemes are not as lightweight as the works mentioned above. Recent progress pushed the round complexity of threshold ECDSA down to two rounds [LLZD25, DGNP25], but uses substantially heavier cryptographic tools (class groups) and considers only static security. For asynchronous settings, [BLSW24] considers adaptive security, but requires more rounds, a suboptimal corruption threshold, and the AGM.

2 Technical Overview

In this section, we provide an overview of our techniques. Essentially, we are not developing any new techniques toward rewinding-free and adaptive security. We are using the techniques from [TSS⁺24] and [KRT24, Che25b], which can be understood as making HBMS [BD21] rewinding-free and HBTS adaptively secure, respectively. The core of our contribution is a simple and elegant modification to HBMS that renders it partially non-interactive. Meanwhile, we preserve most of the structures, so the techniques from [TSS⁺24] and [KRT24, Che25b] remain perfectly applicable. This brings rewinding-free and adaptive security to partially non-interactive schemes for the first time.

2.1 A Recap on HBMS

Let us recall the construction of HBMS [BD21].

- **Setup:** HBMS runs over a cyclic group $\mathbb{G}$ of prime order p with generator g .
- **Key generation:** The secret key $x \leftarrow \mathbb{Z}_p$ is chosen uniformly at random. The public key is $X := g^x$.
- **Key aggregation:** The key aggregation algorithm KAgg outputs $\text{apk} := \prod_{i \in [n]} X_i^{a_i}$, where $a_i := H_a(\mathcal{L}, X_i)$, and $\mathcal{L}$ is the public-key (multi-)set.
- **Signing:** Suppose n users are going to jointly sign on a message μ . They sign as follows:
 - In the first round, each signer (without loss of generality, indexed by 1) uniformly chooses $r_1, w_1 \leftarrow \mathbb{Z}_p$ and broadcasts $R_1 := g^{r_1} h^{w_1}$, where $h := H_h(\text{apk}, \mu)$.
 - In the second round, each signer receives $\{R_i\}_{i \in [n]}$ from other signers. It computes $(z_1, y_1) := (r_1 + a_1 c x_1, w_1)$ for $c := H_c(\text{apk}, \mu, R)$, where $R := \prod_{i \in [n]} R_i$. It broadcasts (z_1, y_1) .
 - Given $\{(z_i, y_i)\}_{i \in [n]}$ from the signers, the final multi-signature is generated as $\sigma := (c, z, y)$, where $z := \sum_{i \in [n]} z_i$ and $y := \sum_{i \in [n]} y_i$.
- **Verification:** The verifier recovers $R := g^z h^y \text{apk}^{-c}$, where $h := H_h(\text{apk}, \mu)$, and checks if $c = H_c(\text{apk}, \mu, R)$.

One way to understand HBMS is a multi-signature version of the Okamoto signature scheme [Oka93], where h is a hash of (apk, μ) rather than a global parameter. Any (α, β) such that $g^\alpha h^\beta = X$ can be used to generate Okamoto signatures. In HBMS, the signer uses $(x, 0)$ as the Okamoto signing key, which works regardless of the choice of h .

Security Proof.

HBMS is proved secure under the DL assumption. The goal of the adversary in the security game is to forge a multi-signature under a public-key set that contains the honest target public key X_* . In the security proof, the reduction sets the target public key X_* to be the input group element. The reduction sets $H_h(\text{apk}, \mu)$ to be g^α with $\alpha \leftarrow \mathbb{Z}_p$ for a partition of (apk, μ) , which we call the “extractable” partition. It sets $H_h(\text{apk}, \mu)$ to be $g^\alpha X_*^\beta$ with $\alpha \leftarrow \mathbb{Z}_p$ and $\beta \neq 0$ for the other partition, which we call the “signable” partition.

For every (apk, μ) from the signable partition, it holds that $g^{-\alpha/\beta} h^{1/\beta} = g^{-\alpha/\beta} \cdot (g^\alpha X_*^\beta)^{1/\beta} = X_*$. Thus, the reduction knows an (apk, μ) -specific Okamoto signing key $(-\alpha/\beta, 1/\beta)$ and can sign (apk, μ) . The reduction expects that the honest signer is only queried to sign (apk, μ) from the signable partition, so it can simulate the security game.

The reduction expects that, on the other hand, the adversary returns a forged signature (c_*, z_*, y_*) for (apk_*, μ_*) from the extractable partition. Let $R_* = g^{z_*} h^{y_*} \text{apk}_*^{-c_*}$, where $h = H_h(\text{apk}_*, \mu_*)$. The validity of the forged signature requires that $H_c(\text{apk}_*, \mu_*, R_*) = c_*$. Since (apk_*, μ_*) lies in the extractable partition, the reduction knows the discrete logarithm α of h . Therefore, we have $R_* = g^{z_* + \alpha y_*} \text{apk}_*^{-c_*}$. Essentially, for the extractable partition, the signature degrades to a Schnorr signature $(R_*, z_* + \alpha y_*)$. The reduction can apply the same (nested) rewinding as in Schnorr multi-signatures [MPSW19, NRS21] to extract the discrete logarithm of X_* .

2.2 From HBMS to HBMS2[DL]: Our Core Technique

The first round of HBMS cannot be preprocessed, as the signer needs to compute h from apk and μ . We resolve this by using another way to derive h from a hash function:

Let u and v be public group elements. We let $h := uv^b$ for $b := H_b(\text{apk}, \mu)$.

The main idea of the security proof for HBMS still applies. The reduction sets $u := g^\alpha X_*^\gamma$ and $v := g^\beta X_*^{-1}$ with $\alpha, \beta, \gamma \leftarrow \mathbb{Z}_p$. It follows that $h = uv^b = g^{\alpha+b\beta} X_*^{\gamma-b}$. The reduction guesses the query $H_b(\text{apk}_*, \mu_*)$ that will correspond to the adversary's forgery and programs $H_b(\text{apk}_*, \mu_*) := \gamma$. Consequently, we have $h = g^{\alpha+b\beta}$, which puts (apk_*, μ_*) to the extractable partition. For any other (apk, μ) , we have $H_b(\text{apk}, \mu) \neq \gamma$ with high probability. Consequently, we have $h = g^{\alpha+b\beta} X_*^{\gamma-b}$ with $\gamma - b \neq 0$, which puts (apk, μ) to the signable partition with a (apk, μ) -specific Okamoto signing key $(-(\alpha + b\beta)/(\gamma - b), 1/(\gamma - b))$. We note that such a partitioning method is inspired by the Boneh-Boyen identity-based encryption and signature scheme [BB04].

Now let us see the magic: h becomes a linear combination of public elements u and v . In the first round, each signer (indexed by 1) can just compute two partial nonces R_1 and S_1 over u and v , respectively. It can defer computing b until the second round and then linearly combine R_1 and S_1 to get its real nonce $R_1 S_1^b$. As a result, the first round becomes independent of the message and the set of signers! Specifically, our HBMS2[DL] is described as follows:

- **Setup:** HBMS2[DL] runs over a cyclic group $\mathbb{G}$ of prime order p with generator g . The public parameters additionally contain uniformly random group elements u and v .
- **Key generation:** Same as in HBMS, the secret key $x \leftarrow \mathbb{Z}_p$ is chosen uniformly at random, and the public key is set to be $X := g^x$.
- **Key aggregation:** Same as in HBMS, the key aggregation algorithm KAgg outputs $\text{apk} := \prod_{i \in [n]} X_i^{a_i}$, where $a_i := H_a(\mathcal{L}, X_i)$, and $\mathcal{L}$ is the public-key (multi-)set.
- **Signing:**
 - In the first round, each signer (without loss of generality, indexed by 1) uniformly chooses $r_1, s_1, w_1 \leftarrow \mathbb{Z}_p$ and broadcasts $R_1 := g^{r_1} u^{w_1}$ and $S_1 := g^{s_1} v^{w_1}$.
 - In the second round, each signer receives $\{(R_i, S_i)\}_{i \in [n]}$ from other signers. The signer computes $R := \prod_{i \in [n]} (R_i S_i^b)$ for $b := H_b(\text{apk}, \mu)$. Then it computes $(z_1, y_1) := (r_1 + bs_1 + a_1 c x_1, w_1)$ for $c := H_c(\text{apk}, \mu, R)$. It broadcasts (z_1, y_1) .
 - Same as in HBMS, given $\{(z_i, y_i)\}_{i \in [n]}$ from the signers, the final multi-signature is generated as $\sigma := (c, z, y)$, where $z := \sum_{i \in [n]} z_i$ and $y := \sum_{i \in [n]} y_i$.
- **Verification:** The verifier recovers $R := g^z (uv^b)^y \text{apk}^{-c}$, where $b := H_b(\text{apk}, \mu)$, and checks if $c = H_c(\text{apk}, \mu, R)$.

Comparison to MuSig2.

Interestingly, HBMS2[DL] has a similar form to MuSig2 [NRS21]. In MuSig2, each signer also outputs multiple partial nonces in the first round. In the second round, the signers derive a coefficient from a hash function to combine the partial nonces into the aggregated nonce.

The key difference is that the combination of partial nonces in MuSig2 is only used within the group of signers but not during verification. In HBMS2[DL], however, b is a public hash of (apk, μ) used in the verification. The hash has the effect of partitioning the space of (apk, μ) in HBMS2[DL] but not in MuSig2. In the next subsection, we will see that partitioning turns out to be important for the rewinding-free technique from [TSS⁺24] to be applicable.

2.3 Toward Rewinding-Free Security

Starting from HBMS2[DL], we obtain our HBMS2[DDH] in a manner analogous to how HBMS is transformed into TSSHO [TSS⁺24]. Specifically, the public parameters g, u, v are replaced with

pairs of elements $\mathbf{g} := (g, \hat{g})$, $\mathbf{u} := (u, \hat{u})$, $\mathbf{v} := (v, \hat{v})$. Throughout the scheme, exponentiation over g , u , v is replaced with element-wise exponentiation over $\mathbf{g}$, $\mathbf{u}$, $\mathbf{v}$, respectively. Consequently, now the public key consists of two elements $\mathbf{X} = (X, \hat{X}) := (g^x, \hat{g}^x)$. Similarly, the aggregated public key and the nonces also consist of two elements. The signature is still in the form of (c, z, y) . To verify it, one recovers $\mathbf{R} := \mathbf{g}^z(\mathbf{u}\mathbf{v}^b)^y \mathbf{apk}^{-c}$ and checks if $c = H_c(\mathbf{apk}, \mu, \mathbf{R})$, where multiplication is element-wise.

In the security proof, we still set $\mathbf{u} := \mathbf{g}^\alpha \mathbf{X}_*^\gamma$ and $\mathbf{v} := \mathbf{g}^\beta \mathbf{X}_*^{-1}$ with $\alpha, \beta, \gamma \leftarrow \mathbb{Z}_p$, where $\mathbf{X}_*$ is the target public key. By DDH, we indistinguishably switch to a game where the target public key $\mathbf{X}_*$ is independent of $\mathbf{g}$, which makes a valid secret key x unlikely to exist, i.e., $\mathbf{X}_* \notin \text{Span}\{\mathbf{g}\}$. For any $(\mathbf{apk}, \mu)$ in the signable partition, the reduction can still sign using the $(\mathbf{apk}, \mu)$ -specific Okamoto signing key $(-(\alpha + b\beta)/(\gamma - b), 1/(\gamma - b))$.

On the other hand, the extractable partition now becomes the “unsignable partition”. A valid forgery (c_*, z_*, y_*) for $(\mathbf{apk}_*, \mu_*)$ requires $H_c(\mathbf{apk}_*, \mu_*, \mathbf{R}_*) = c_*$ for $\mathbf{R}_* := \mathbf{g}^{z_*}(\mathbf{u}\mathbf{v}^b)^{y_*} \mathbf{X}_*^{-c_*}$ and $b := H_b(\mathbf{apk}_*, \mu_*)$. In this partition, we have $b = \gamma$ and therefore $\mathbf{u}\mathbf{v}^b = \mathbf{g}^{\alpha+\beta\gamma}$. Consequently, a valid forgery has to satisfy $\mathbf{R}_* \mathbf{X}_*^{c_*} \in \text{Span}\{\mathbf{g}\}$, since $\mathbf{R}_* \mathbf{X}_*^{c_*} = \mathbf{g}^{z_*}(\mathbf{u}\mathbf{v}^b)^{y_*} = \mathbf{g}^{z_*+(\alpha+\beta\gamma)y_*}$. However, since $\mathbf{X}_* \notin \text{Span}\{\mathbf{g}\}$ and c_* is the uniformly random value of $H_c(\mathbf{apk}_*, \mu_*, \mathbf{R}_*)$, $\mathbf{R}_* \mathbf{X}_*^{c_*} \in \text{Span}\{\mathbf{g}\}$ is unlikely to hold. This means that, in this game with a fake public key $\mathbf{X}_*$, making a forgery for $(\mathbf{apk}_*, \mu_*)$ is unconditionally hard with a bounded number of queries to H_c . Thus, making a forgery is computationally hard under DDH in the original security game. We have obtained a rewinding-free security proof, with a security loss of only $\Theta(q_h)$ due to guessing the H_b query $H_b(\mathbf{apk}_*, \mu_*)$ that will correspond to the forgery.

The Challenge of Applying Tight Security Techniques to Previous Schemes.

We discuss a question one may ask: why we cannot directly apply the technique in [TSS⁺24] to existing partially non-interactive schemes?

Tracing back, this technique for tighter security proofs follows the high-level idea by Katz and Wang [KW03], which gives a Schnorr-like tightly secure signature scheme based on DDH. Briefly speaking, this idea is to let the scheme have a “lossy” mode in addition to a normal mode, such that these two modes are computationally indistinguishable, and signing is unconditionally hard in the lossy mode. The idea has been generalized to give signature schemes from different assumptions [AFLT16] and underlies all the previous rewinding-free threshold and multi-signature schemes in pairing-free groups [FH21, PW23, PW24, TSS⁺24, Che25a, BW25, BW26, LÖ25].

A key point in the security proofs is that, the efficient reduction needs to be able to perform the security game in the lossy mode, where it is requested to sign, despite the fact that the lossy mode is “unsignable”. This requires the reduction to have a special capability in signing beyond that of a normal user. For (non-interactive) standard signature schemes, the special capability is the honest-verifier zero-knowledge (HVZK) simulators enabled by programming the random oracle. However, such simulators can only output a whole signature in one shot. They are not directly applicable to two-round threshold and multi-signature schemes, where the two components of signatures are output in two rounds.

Therefore, all the rewinding-free two-round threshold and multi-signature schemes [PW23, PW24, TSS⁺24, Che25a, BW25] crucially rely on partitioning. Some parameters are generated by hashing $(\mathbf{apk}, \mu)$, such that the scheme is only lossy for a partition of $(\mathbf{apk}, \mu)$ (corresponding to our unsignable partition) while still signable for other $(\mathbf{apk}, \mu)$. The reduction only signs the signable partition.

In contrast, existing partially non-interactive two-round schemes [KG20, AB21, NRS21, TZ23] do not do partitioning. The reductions rely on interactive assumptions and use their own oracles to simulate signing. This requires that, in the game that the reduction executes, signing is only computationally hard. If we apply the Katz-Wang technique and switch to a lossy mode where signing is unconditionally hard, then even oracles cannot be helpful. We successfully overcome this

barrier by introducing a novel partitioning technique that allows to delay hashing to the second round.

2.4 Toward Adaptive Security

We first briefly explain how to turn our HBMS2[DDH] into a (statically secure) threshold scheme HBTS2[DDH]. Suppose that we have a group of n signers with threshold t . Instead of letting each signer generate its own key, we now have a centralized key generation. It samples the main secret key $x \leftarrow \mathbb{Z}_p$ and distributes it using Shamir's secret sharing as $\{x_i\}_{i \in [n]}$. For every signing set $\mathcal{S} \subseteq [n]$ where $|\mathcal{S}| = t + 1$, the main secret key can be recovered as $x = \sum_{i \in \mathcal{S}} \lambda_{\mathcal{S},i} x_i$, where $\lambda_{\mathcal{S},i}$ is the corresponding Lagrange coefficient. The main public key is $\mathbf{X} := \mathbf{g}^x$. To jointly generate a signature under the main key, signers in $\mathcal{S}$ run the same protocol as in the multi-signature scheme, except that the key aggregation coefficient a_i is replaced with $\lambda_{\mathcal{S},i}$.

In the security proof, again, $\mathbf{X}$ is uniformly chosen such that $\mathbf{X} \notin \text{Span}\{\mathbf{g}\}$ with high probability. Without a valid secret key, the key generation can be simulated by performing Shamir's secret sharing on exponents, which outputs public key shares $\{\mathbf{X}_i\}_{i \in [n]}$ such that $\mathbf{X} = \prod_{i \in \mathcal{S}} \mathbf{X}_i^{\lambda_{\mathcal{S},i}}$ for every $\mathcal{S} \subseteq [n]$, $|\mathcal{S}| = t + 1$. As a result, there exist at most t public key shares $\mathbf{X}_i$ such that the corresponding secret key shares x_i exist.

This causes the main issue for adaptive security. An adaptive adversary can corrupt any t signers during the security game. Since there exist at most t secret key shares, the reduction has to guess who the adversary will corrupt, which introduces an exponential security loss of $\binom{n}{t}$.

We now introduce our HBTS2[DDH]-Mask, which applies the masking technique from [KRT24, Che25b] to HBTS2[DDH]. In HBTS2[DDH], the public key share $\mathbf{X}_i$ becomes public once the adversary requests a signature share from signer i . The public key share $\mathbf{X}_i$ binds the corresponding secret key share x_i , so the reduction has to correctly output x_i to simulate the game. The main idea of masking is to mask the signature shares by one-time shares of zero. As a result, the public key shares are never revealed, and the secret key shares become perfectly hidden until being output for corruptions. The reduction does not need to simulate the key sharing and can simply sample one uniformly when asked to output a secret key share.

[Che25b] observed that for HBTS to achieve non-AGM adaptive security, only the second round needs to be masked. This turns out to be also true for our HBTS2. To make sure the masks are indeed one-time, they are generated by hashing the signers' view in the signing protocol, which is of high entropy.

In comparison, if we only mask the second round of FROST, we only achieve adaptive security in the AGM [BCL⁺25, BGC⁺25]. If we mask both rounds of FROST, we indeed obtain non-AGM adaptive security [Che25b]; however, for the first-round masks to be one-time, we either require the signers to be stateful or to spend one more round to jointly generate some entropy first.

3 Preliminaries

Notations.

For a positive integer n , $[n]$ denotes $\{1, 2, \dots, n\}$. For a randomized algorithm A , $y \in A(x)$ denotes that y is a possible output of A on input x . For vectors $\mathbf{g}, \mathbf{h}$ of group elements, exponentiation $\mathbf{g}^x$ and multiplication $\mathbf{gh}$ are both element-wise.

Shamir's Secret Sharing.

We use Shamir's secret sharing scheme [Sha79] to share a secret over $\mathbb{Z}_p$. On input the number of users $n < p$, the threshold $t \leq n$, and the secret $x \in \mathbb{Z}_p$, the sharing algorithm $\text{Share}(n, t, x)$ sets $\alpha_0 := x$ and samples the other t coefficients $\alpha_1, \dots, \alpha_t \leftarrow \mathbb{Z}_p$ of polynomial $\alpha(\cdot)$. It returns

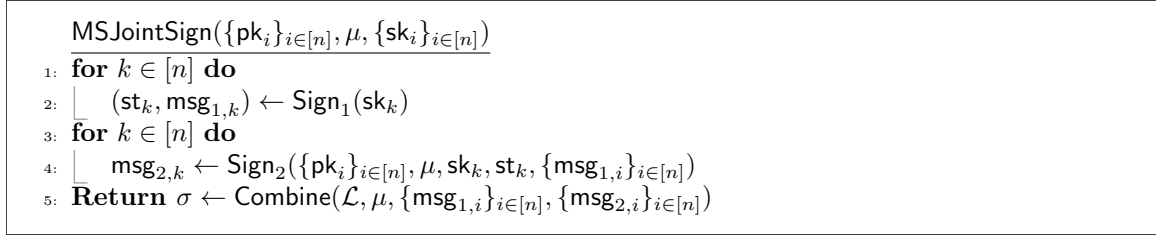


Figure 1: Procedure MSJointSign used to define the completeness of multi-signature schemes.

the secret shares $\{x_i\}_{i \in [n]}$ where $x_i = \alpha(i) = \sum_{j=0}^t \alpha_j i^j$. For any $(t+1)$ -subset $\mathcal{S}$ of $[n]$, we have $x = \sum_{i \in \mathcal{S}} \lambda_{\mathcal{S},i} x_i$ by polynomial interpolation, where the Lagrange coefficient for $\mathcal{S}$ is defined as $\lambda_{\mathcal{S},i} = \prod_{j \in \mathcal{S}, j \neq i} j/(j-i)$. The security of Shamir's secret sharing guarantees that the collection of any t shares of x is independent of x .

DDH Assumption.

Let GrGen be a group generation algorithm that takes the security parameter 1^λ as input and outputs group parameters $(\mathbb{G}, p, g)$, where $\mathbb{G}$ is a cyclic group of prime order p and g is a generator.

Definition 1 (Decisional Diffie-Hellman (DDH)). We say that DDH is (τ, ε) -hard with respect to GrGen if for every positive integer λ , every τ -time algorithms A ,

$$\begin{aligned} & |\Pr[A(\mathbb{G}, p, g, \hat{g}, g^a, \hat{g}^a) = 1 : (\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda), \hat{g} \leftarrow \mathbb{G}, a \leftarrow \mathbb{Z}_p] \\ & - \Pr[A(\mathbb{G}, p, g, \hat{g}, g^a, \hat{g}^b) = 1 : (\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda), \hat{g} \leftarrow \mathbb{G}, a, b \leftarrow \mathbb{Z}_p] = 1| \leq \varepsilon. \end{aligned}$$

Let $(g, \hat{g}, h, \hat{h})$ be a tuple of four group elements over $\mathbb{G}$ where g is a generator. We call it a *DH-tuple* if $(h, \hat{h}) \in \text{Span}\{(g, \hat{g})\}$.

3.1 Multi-Signature Schemes

Syntax. We define partially non-interactive two-round multi-signature schemes with key aggregation. We follow the plain public-key model [BN06]. Each user's key pair is individually generated by the key generation algorithm KGen. A deterministic, non-interactive key aggregation KAgg can aggregate a set of individual public keys into a single aggregated key, such that the multi-signature verification algorithm Vf can take only the aggregated public key rather than the whole public key list as input. The two-round signing protocol consists of three algorithms: each signer runs Sign₁ and Sign₂ to generate its first and second-round protocol messages to broadcast, respectively. The last algorithm Combine generates the final multi-signature from the whole protocol transcript, which is deterministic and can be executed by any designated user. The partially non-interactive property is reflected in that Sign₁ does not take the public key set and the message as inputs. We use procedure MSJointSign in Fig. 1 to describe an honest execution of the signing protocol and also to define the completeness.

We remark that the protocol is independent of the indexing of signers, public keys, and protocol messages. That is, the signers do not need to agree on an order among them, as any order will yield the same output. Without loss of generality, we assume that each user indexes itself by 1.

Definition 2 (Partially non-interactive two-round multi-signatures with key aggregation). A *partially non-interactive two-round multi-signature scheme MS with key aggregation* consists of algorithms with syntax defined as follows:

- **Setup** $(1^\lambda) \rightarrow \text{pp}$: The setup algorithm takes the security parameter 1^λ as input and outputs the public parameter **pp**. All algorithms related to MS implicitly take **pp** as input.

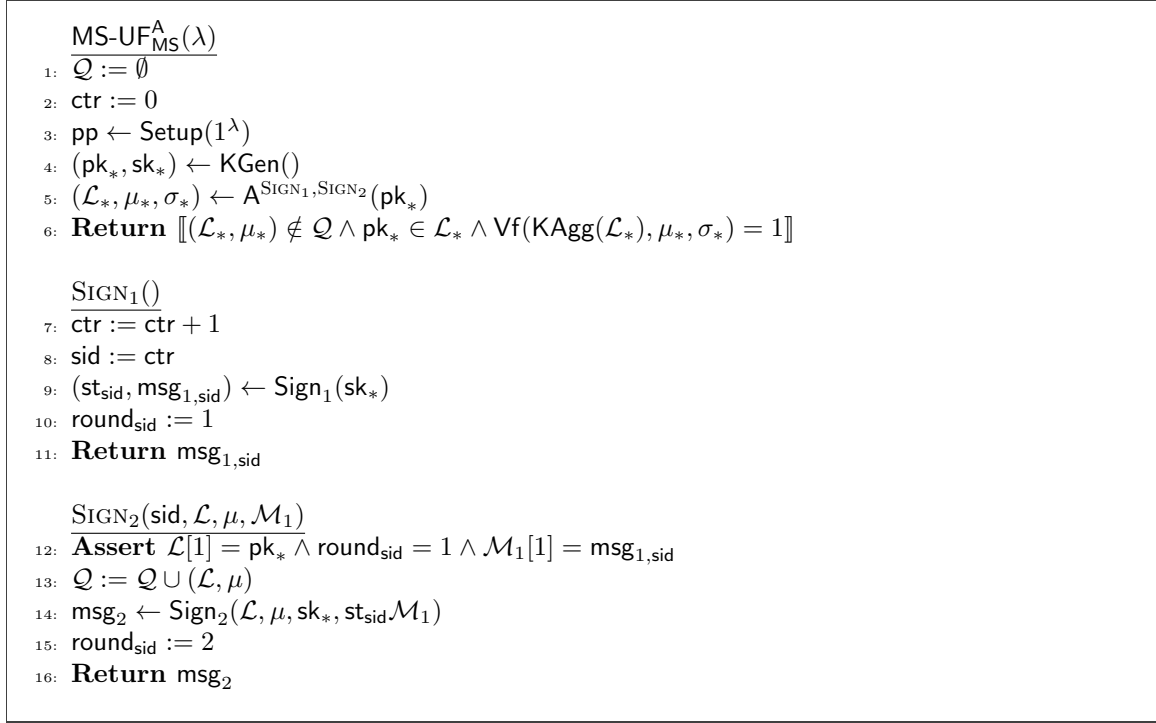


Figure 2: The MS-UF security game for multi-signature scheme MS against A.

- $\text{KGen}() \rightarrow (\text{pk}, \text{sk})$: The key generation algorithm outputs a secret key sk and a public key pk .
 - $\text{KAgg}(\mathcal{L}) \rightarrow \text{apk}$: The deterministic key aggregation algorithm takes a set of public keys $\mathcal{L} = \{\text{pk}_1, \dots, \text{pk}_n\}$ as input and outputs an aggregated public key apk .
 - The signing protocol consists of three algorithms:
 - $\text{Sign}_1(\text{sk}_1) \rightarrow (\text{st}_1, \text{msg}_{1,1})$: The first-round signing algorithm takes a secret key sk_1 as input and outputs a state st_1 and a first-round protocol message $\text{msg}_{1,1}$.
 - $\text{Sign}_2(\mathcal{L}, \mu, \text{sk}_1, \text{st}_1, \mathcal{M}_1) \rightarrow \text{msg}_{2,1}$: The second-round signing algorithm takes a set of public keys $\mathcal{L}$, a message μ , and a secret key sk_1 , a state st_1 , and a set of first-round protocol messages $\mathcal{M}_1$ as inputs and outputs a second-round protocol message $\text{msg}_{2,1}$.
 - $\text{Combine}(\mathcal{L}, \mu, \mathcal{M}_1, \mathcal{M}_2) \rightarrow \sigma$: The combining algorithm takes a set of public key $\mathcal{L}$, a message μ , a set of first-round protocol messages $\mathcal{M}_1$ and a set of second-round protocol messages $\mathcal{M}_2$ as inputs and outputs a multi-signature σ .
 - $\text{Vf}(\text{apk}, \mu, \sigma) \rightarrow 0/1$: The deterministic verification algorithm takes an aggregated public key apk , a message μ , and a multi-signature σ as inputs and outputs 0 or 1.
- For completeness, we require that for every λ , every $\text{pp} \in \text{Setup}(1^\lambda)$, every n , every $(\text{pk}_1, \text{sk}_1), \dots, (\text{pk}_n, \text{sk}_n) \in \text{KGen}()$, and every message μ , we have

$$\Pr[\text{Vf}(\text{KAgg}(\{\text{pk}_i\}_{i \in [n]}), \mu, \sigma) = 1 : \sigma \leftarrow \text{MSJointSign}(\{\text{pk}_i\}_{i \in [n]}, \mu, \{\text{sk}_i\}_{i \in [n]})] = 1,$$

where the procedure MSJointSign , modeling an honest execution of MS, is defined in Fig. 1.

Security. The security of two-round multi-signature schemes with key aggregation is defined by the security game MS-UF described in Fig. 2. The adversary A is given the public parameter pp and a target public key pk_* . Its goal is to forge a multi-signature on a message μ_* under a public-key set $\mathcal{L}_*$ that contains pk_* , both of its own choice. A can concurrently open multiple signing sessions

```

    TSJointSign(pk, S, μ, {ski}i∈S)
1: for k ∈ S do
2:   (stk, msg1,k) ← Sign1(pk, k, skk)
3: for k ∈ S do
4:   msg2,k ← Sign2(pk, S, μ, k, skk, stk, {msg1,i}i∈S)
5: Return σ ← Combine(pk, S, μ, {msg1,i}i∈S, {msg2,i}i∈S)

```

Figure 3: Procedure TSJointSign used to define the completeness of threshold signature schemes.

with an honest signer possessing pk_* . In each session, A chooses the message μ to sign, and it plays the part of all other signers, with public-key set $\mathcal{L}$ of its choice. This is formalized by two signing oracles SIGN_1 , SIGN_2 , where we use a session identifier sid to distinguish the concurrent sessions. It is required that no signing session on $(\mathcal{L}_*, \mu_*)$ has been completed for the forgery to be non-trivial.

Definition 3 (Security of Two-Round Multi-Signature Schemes with Key Aggregation). A two-round multi-signature scheme MS with key aggregation is (q_s, τ, ε) -secure if for every λ , every adversary A that makes at most $q_s = q_s(\lambda)$ queries to SIGN_1 and runs in time at most $\tau = \tau(\lambda)$, $\Pr[\text{MS-UF}_{\text{MS}}^A(\lambda) = 1] \leq \varepsilon = \varepsilon(\lambda)$.

3.2 Threshold Signature Schemes

Syntax. We define partially non-interactive two-round threshold signature schemes. A centralized key generation algorithm KGen generates a public key and n secret key shares. The two-round signing protocol consists of three algorithms Sign_1 , Sign_2 , Combine as in multi-signature schemes. An honest execution of the signing protocol is described as the procedure TSJointSign in Fig. 3, which we use to define completeness.

Definition 4 (Partially non-interactive two-Round Threshold Signature Schemes). A *partially non-interactive two-round threshold signature scheme* TS consists of the following algorithms:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: On input the security parameter 1^λ , the setup algorithm Setup outputs global parameters pp. All algorithms related to TS implicitly take pp as input.
- $\text{KGen}(n, t) \rightarrow (\text{pk}, \{\text{sk}_i\}_{i \in [n]})$: On inputs a number of users n and a threshold t , the key generation algorithm KGen outputs a public key pk and n secret key shares $\{\text{sk}_i\}_{i \in [n]}$.
- The signing protocol consists of three algorithms:
 - $\text{Sign}_1(\text{pk}, k, \text{sk}_k) \rightarrow (\text{st}_k, \text{msg}_{1,k})$: On inputs a public key pk, a signer index k , a secret key share sk_k , the first-round signing algorithm Sign_1 outputs a state st_k and a first-round protocol message $\text{msg}_{1,k}$.
 - $\text{Sign}_2(\text{pk}, S, \mu, k, \text{sk}_k, \text{st}_k, \mathcal{M}_1) \rightarrow \text{msg}_{2,k}$: On inputs a public key pk, a set of signer indices S , a message μ , a signer index k , a secret key share sk_k , a state st_k , and a set of first-round protocol messages $\mathcal{M}_1$, the second-round signing algorithm Sign_2 outputs a second-round protocol message $\text{msg}_{2,k}$.
 - $\text{Combine}(\text{pk}, S, \mu, \mathcal{M}_1, \mathcal{M}_2) \rightarrow \sigma$: On inputs a public key pk, a set of signer indices S , a message μ , a set of first-round protocol messages $\mathcal{M}_1$, and a set of second-round protocol messages $\mathcal{M}_2$, the combining algorithm outputs a signature σ .
- $\text{Vf}(\text{pk}, \mu, \sigma) \rightarrow 0/1$: On inputs a public key pk, a message μ , and a signature σ , the deterministic verification algorithm Vf outputs 0 or 1.

For completeness, we require that for every λ , every $\text{pp} \in \text{Setup}(1^\lambda)$, every n and t that TS supports, every $(\text{pk}, \{\text{sk}_i\}_{i \in [n]}) \in \text{KGen}(n, t)$, every subset of signers $S \subseteq [n]$ with $|S| = t + 1$, and every message μ , we have

$$\Pr[\text{Vf}(\text{pk}, \mu, \sigma) = 1 : \sigma \leftarrow \text{TSJointSign}(\text{pk}, S, \mu, \{\text{sk}_i\}_{i \in S})] = 1,$$

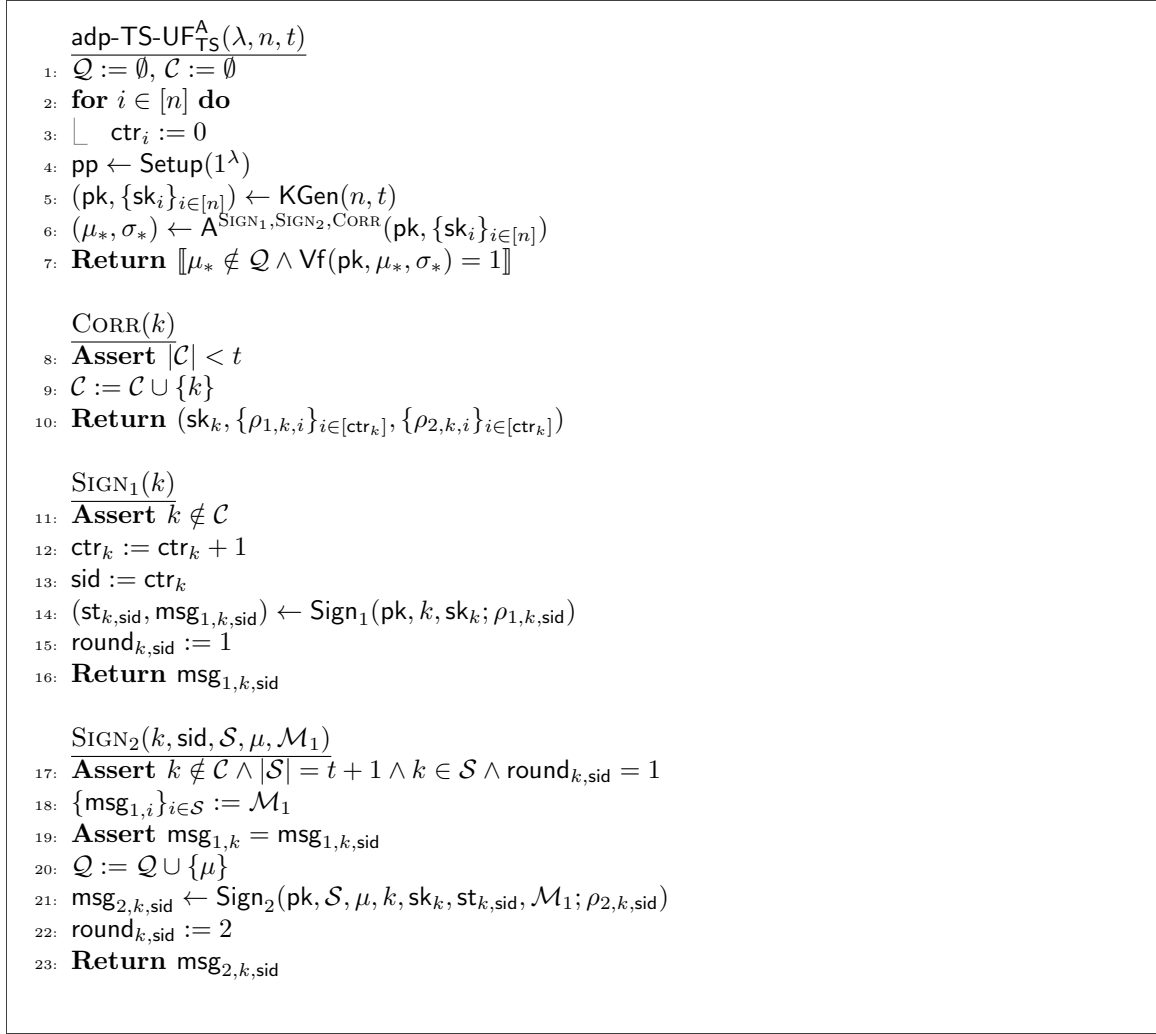


Figure 4: The adp-TS-UF security game.

where the procedure TSJointSign, modeling an honest execution of TS, is defined in Fig. 3.

Security. The fully adaptive security of two-round threshold signature schemes is defined by the security game adp-TS-UF described in Fig. 4. Our definition is the adaptive version of the TS-UF-0 security in [BCK⁺22]. The adversary $\mathbf{A}$ is given the public parameter pp , the public key pk , and the access to the signing and corruption oracles. Its goal is to forge a signature on a message that it has never queried to the signing oracle. It can concurrently open multiple signing sessions with each users, and we use a session identifier sid to distinguish the concurrent sessions with each user. $\mathbf{A}$ can corrupt at most t users in total, each corruption at any time and adaptively based on its previous view.

Our model does not assume secure erasure. When a signer is corrupted, the adversary not only learns its secret key share but also the *whole computation history*. Our corruption oracle returns all randomness that the signer used in previous signing sessions, which allows the adversary to recover the whole history.

Our model considers fully corrupted communication channels. As the inputs to the second-round

<u>Setup(1^λ)</u> 1: $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda)$ 2: $\hat{g}, u, \hat{u}, v, \hat{v} \leftarrow \mathbb{G}$ 3: $\mathbf{g} := (g, \hat{g}), \mathbf{u} := (u, \hat{u}), \mathbf{v} := (v, \hat{v})$ 4: Sample $H_a, H_b, H_c : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ 5: Return $(\mathbb{G}, p, \mathbf{g}, \mathbf{u}, \mathbf{v}, H_a, H_b, H_c)$ <u>KGen()</u> 6: $x \leftarrow \mathbb{Z}_p$ 7: $\mathbf{X} := \mathbf{g}^x$ 8: Return $(x, \mathbf{X})$ <u>KAgg($\mathcal{L}$)</u> 9: $\{\mathbf{X}_i\}_{i \in [n]} := \mathcal{L}$ 10: for $i \in [n]$ do 11: $a_i := H_a(\mathcal{L}, \mathbf{X}_i)$ 12: $\tilde{\mathbf{X}} := \prod_{i \in [n]} \mathbf{X}_i^{a_i}$ 13: Return $\tilde{\mathbf{X}}$ <u>Sign₁(sk₁)</u> 14: $r_1, s_1, w_1 \leftarrow \mathbb{Z}_p$ 15: $\mathbf{R}_1 := \mathbf{g}^{r_1} \mathbf{u}^{w_1}$ 16: $\mathbf{S}_1 := \mathbf{g}^{s_1} \mathbf{v}^{w_1}$ 17: Return $((r_1, s_1, w_1), (\mathbf{R}_1, \mathbf{S}_1))$ <u>Sign₂($\mathcal{L}, \mu, \text{sk}_1, \text{st}_1, \mathcal{M}_1$)</u> 18: $\text{apk} := \text{KAgg}(\mathcal{L})$ 19: $\{\mathbf{X}_i\}_{i \in [n]} := \mathcal{L}$	20: $a_1 := H_a(\mathcal{L}, \text{pk}_1)$ 21: $x_1 := \text{sk}_1$ 22: $(r_1, s_1, w_1) := \text{st}_1$ 23: $\{(\mathbf{R}_i, \mathbf{S}_i)\}_{i \in [n]} := \mathcal{M}_1$ 24: $b := H_b(\text{apk}, \mu)$ 25: $\mathbf{R} := \prod_{i \in [n]} (\mathbf{R}_i \mathbf{S}_i^b)$ 26: $c := H_c(\text{apk}, \mu, \mathbf{R})$ 27: $z_1 := r_1 + b s_1 + a_1 c x_1$ 28: $y_1 := w_1$ 29: Return (z_1, y_1) <u>Combine($\mathcal{L}, \mu, \mathcal{M}_1, \mathcal{M}_2$)</u> 30: $\text{apk} := \text{KAgg}(\mathcal{L})$ 31: $\{(\mathbf{R}_i, \mathbf{S}_i)\}_{i \in [n]} := \mathcal{M}_1$ 32: $\{(z_i, y_i)\}_{i \in [n]} := \mathcal{M}_2$ 33: $b := H_b(\text{apk}, \mu)$ 34: $\mathbf{R} := \prod_{i \in [n]} (\mathbf{R}_i \mathbf{S}_i^b)$ 35: $c := H_c(\text{apk}, \mu, \mathbf{R})$ 36: $(z, y) := \sum_{i \in [n]} (z_i, y_i)$ 37: Return (c, z, y) <u>Vf(apk, μ, σ)</u> 38: $\tilde{\mathbf{X}} := \text{apk}$ 39: $(c, z, y) := \sigma$ 40: $b := H_b(\text{apk}, \mu)$ 41: $\mathbf{R} := \mathbf{g}^z (\mathbf{u} \mathbf{v}^b)^y \tilde{\mathbf{X}}^{-c}$ 42: Return $\llbracket c = H_c(\text{apk}, \mu, \mathbf{R}) \rrbracket$
--	--

Figure 5: HBMS2[DDH], defined with respect to group generation algorithm GrGen.

signing oracles, the adversary decides every protocol message even on behalf of honest parties. In fact, honest signers will not realize the signing sessions of others. There is no consistency requirement on the protocol messages chosen by the adversary.

Definition 5 (Adaptive Security of Threshold Signature Schemes.). A two-round threshold signature scheme TS is (q_s, τ, ε) -adaptively secure if for every λ , every adversary $\mathbf{A}$ that makes at most $q_s = q_s(\lambda)$ signing queries and runs in time at most $\tau = \tau(\lambda)$, for every n and t that TS supports, $\Pr[\text{adp-TS-UF}_{\text{TS}}^{\mathbf{A}}(\lambda, n, t) = 1] \leq \varepsilon = \varepsilon(\lambda)$, where the security game adp-TS-UF is defined in Fig. 4.

4 HBMS2[DDH]: Multi-Signature Scheme with Tighter Security

4.1 Scheme

We describe HBMS2[DDH], defined with respect to group generation algorithm GrGen, in Fig. 5.

Completeness. The following equation shows that, for an honestly generated multi-signature, the verification correctly recovers the aggregated commitment. Then the completeness follows directly.

$$\begin{aligned}
\mathbf{g}^z(\mathbf{uv}^b)^y \tilde{\mathbf{X}}^{-c} &= \mathbf{g}^{\sum_{i \in [n]} (r_i + bs_i + a_i cx_i)} (\mathbf{uv}^b)^{\sum_{i \in [n]} w_i} \left(\prod_{i \in [n]} \mathbf{X}_i^{a_i} \right)^{-c} \\
&= \prod_{i \in [n]} (\mathbf{g}^{r_i + bs_i + a_i cx_i} \mathbf{u}^{w_i} \mathbf{v}^{bw_i} \mathbf{g}^{-a_i cx_i}) \\
&= \prod_{i \in [n]} (\mathbf{g}^{r_i} \mathbf{u}^{w_i} \mathbf{g}^{bs_i} \mathbf{v}^{bw_i}) \\
&= \prod_{i \in [n]} (\mathbf{R}_i \mathbf{S}_i^b) = \mathbf{R}.
\end{aligned}$$

4.2 Security

Theorem 1. *If DDH is $(\tau_{\text{ddh}}, \varepsilon_{\text{ddh}})$ -hard for GrGen, then HBMS2[DDH] is $(q_s, \tau_{\text{uf}}, \varepsilon_{\text{uf}})$ -MS-UF in the random oracle model against any adversary that makes at most q_h hash queries, where*

$$\varepsilon_{\text{ddh}} = \frac{\varepsilon_{\text{uf}}}{q' + 1} - \frac{4q + 2}{p},$$

$q = n_{\text{max}}(q_s + 1) + q_h$, $q' = q_s + q_h + 1$, $n_{\text{max}} < p$ is the maximal size of public-key sets that $\mathbf{A}$ returns or queries, and $\tau_{\text{uf}} \approx \tau_{\text{ddh}}$.

Proof. Let $\mathbf{A}$ be any τ_{uf} -time adversary that makes at most q_s queries to SIGN_1 and q_h hash queries to each hash oracle. We need to prove that $\mathbf{A}$'s winning probability is at most ε_{uf} . To do so, we will build a series of hybrid games. Let $\mathbf{G}_0$ be the original $\text{MS-UF}_{\text{HBMS2[DDH]}}^{\mathbf{A}}$ game. Eventually, we will reach a game where we can unconditionally bound $\mathbf{A}$'s winning probability. Let ε_i denote the probability that $\mathbf{A}$ wins $\mathbf{G}_i$. Then we have $\varepsilon_0 = \varepsilon_{\text{uf}}$.

In $\mathbf{G}_0$, let $\mathbf{X}_*$ denote the target public key input to $\mathbf{A}$, and x_* denote the corresponding secret key. Hash functions $\mathbf{H}_a$, $\mathbf{H}_b$, $\mathbf{H}_c$ are modeled as random oracles. We assume that $\mathbf{A}$ never makes repeated queries to a random oracle. This is without loss of generality, as we can easily build a wrapper to ensure so.

It is not hard to see that the total number of queries to $\mathbf{H}_a$, including external queries from $\mathbf{A}$ and internal queries from other oracles and the verification of the final forgery, is bounded by q . The total numbers of queries to $\mathbf{H}_b$ and $\mathbf{H}_c$, respectively, are bounded by q' . One can verify that these bounds hold for every hybrid game in the following.

$\mathbf{G}_1$: Rule out collisions. In this game, we exclude collisions in $\mathbf{H}_b$ and KAgg . Specifically:

- Before $\mathbf{H}_b$ returns, the game aborts if the answer to be returned is repeated with a previous distinct $\mathbf{H}_b$ query.
- For each public key $[\mathcal{L}]$ with $\mathbf{X}_* \in \mathcal{L}$, when $\text{KAgg}(\mathcal{L})$ is determined, i.e. $\mathbf{H}_a(\mathcal{L}, \mathbf{X}_i)$ has been defined for every $i \in |\mathcal{L}|$, this game records $\text{KAgg}(\mathcal{L})$. The game aborts when there exist $\mathcal{L} \neq \mathcal{L}'$ that both contain $\mathbf{X}_*$ such that $\text{KAgg}(\mathcal{L}) = \text{KAgg}(\mathcal{L}')$.

The collision probability in $\mathbf{H}_b$ is clearly upper-bounded by $(q')^2/p < q^2/p$. Below we bound the collision probability for KAgg .

We analyze the abort probability conditioned on $\mathbf{X}_* \neq \mathbb{1}^2$, which is the case except with probability $1/p$. Suppose $\mathbf{X}_*$ appears n_* times in $\mathcal{L}$, and without loss of generality assume $\mathcal{L}[1] = \mathbf{X}_*$. Then $\text{KAgg}(\mathcal{L}) = \mathbf{X}_*^{n_* a_1} \prod_{\mathbf{X}_i \neq \mathbf{X}_* \in \mathcal{L}} \mathbf{X}_i^{a_i}$. Let $\mathbf{Y} := \prod_{\mathbf{X}_i \neq \mathbf{X}_* \in \mathcal{L}} \mathbf{X}_i^{a_i}$. Since $n_* < p$, we have $\mathbf{X}_*^{n_*} \neq \mathbb{1}$. Then, as a_1 is uniform and independent of $\mathbf{Y}$, we have $\text{KAgg}(\mathcal{L})$ uniformly distributed over the coset $\mathbf{Y} \cdot \text{Span}\{\mathbf{X}_*\}$ conditioned on any $\mathbf{Y}$. This means that $\text{KAgg}(\mathcal{L})$ is uniformly distributed over a set

of size at least p . Consequently, $\text{KAgg}(\mathcal{L}) = \text{KAgg}(\mathcal{L}')$ with probability at most $1/p$ for every pair of distinct $\mathcal{L}, \mathcal{L}'$. In total, the collision probability of KAgg is at most $(q^2 + 1)/p$.

Overall, we have $\varepsilon_1 \geq \varepsilon_0 - (2q^2 + 1)/p$.

G₂: Generate $\mathbf{u}$ and $\mathbf{v}$ as combinations of $\mathbf{g}$ and $\mathbf{X}_*$. This game changes the way to generate $\mathbf{u}$ and $\mathbf{v}$ in Setup as follows. It first samples $\mathbf{g}$ and $\mathbf{X}_*$ as in G_1 . Then it samples $\alpha, \beta, \gamma \leftarrow \mathbb{Z}_p$ and computes $\mathbf{u} := \mathbf{g}^\alpha \mathbf{X}_*^\gamma$ and $\mathbf{v} := \mathbf{g}^\beta \mathbf{X}_*^{-1}$.

In G_1 , $\mathbf{u}$ and $\mathbf{v}$ are uniformly distributed over $\mathbb{G}^2$, while in G_2 , $\mathbf{u} = \mathbf{g}^{\alpha+\gamma x_*}$ and $\mathbf{v} = \mathbf{g}^{\beta-x_*}$ are uniformly over $\text{Span}\{\mathbf{g}\}$. We can reduce DDH to distinguishing G_1 and G_2 as follows.

On inputs $\mathbb{G}, p, g, A, B, C$, the reduction runs G_1/G_2 with public parameters $\text{pp} = (\mathbb{G}, p, \mathbf{g}, \mathbf{u}, \mathbf{v})$, where $\mathbf{g} := (g, A)$, $\mathbf{u}$ and $\mathbf{v}$ being uniformly random linear combinations of (g, A) and (B, C) . If (g, A, B, C) is a uniform DH tuple, then $\mathbf{u}$ and $\mathbf{v}$ are both uniformly distributed over $\text{Span}\{\mathbf{g}\}$, so the reduction simulates G_2 . If (A, B, C) are uniformly distributed over $\mathbb{G}^3$, then (g, A, B, C) is not a DH tuple except with probability $1/p$, in which case the reduction simulates G_1 . Therefore, we have $\varepsilon_2 \geq \varepsilon_1 - \varepsilon_{\text{ddh}} - 1/p$.

G₃: Guess the index of $H_b(\text{apk}_*, \mu_*)$. In this game, we change the winning condition of A. In the setup phase, the game uniformly chooses a secret index $i_* \leftarrow [q']$. We perform one more check on A's output $(\mathcal{L}_*, \mu_*, \sigma_*)$ in addition to the previous winning conditions. For A to win, we require that $H_b(\text{KAgg}(\mathcal{L}_*), \mu_*)$ is the i_* -th distinct query made to H_b . Since i_* is independent of A's view, we have $\varepsilon_3 = \varepsilon_2/q'$.

G₄: Abort in advance if impossible to win. This game aborts whenever it turns out that A is impossible to win. In particular, this is when A queries $(\mathcal{L}, \mu)$ to SIGN_2 such that $H_b(\text{KAgg}(\mathcal{L}), \mu)$ is the i_* -th distinct H_b query. Recall that the winning condition requires $(\mathcal{L}_*, \mu_*) \neq (\mathcal{L}, \mu)$, and we have ruled out KAgg collisions in G_1 . Thus, we have $(\text{KAgg}(\mathcal{L}_*), \mu_*) \neq (\text{KAgg}(\mathcal{L}), \mu)$. Consequently, if $H_b(\text{KAgg}(\mathcal{L}), \mu)$ is the i_* -th distinct query, then $H_b(\text{KAgg}(\mathcal{L}_*), \mu_*)$ cannot be the i_* -th query, and A is impossible to win.

This can be checked by tracking the indices of H_b queries. Recall that when A queries $(\mathcal{L}, \mu)$ to SIGN_2 , the game makes an internal query $H_b(\text{KAgg}(\mathcal{L}), \mu)$ when executing Sign_2 . At this time, we let the game abort if this $H_b(\text{KAgg}(\mathcal{L}), \mu)$ is the i_* -th distinct H_b query.

Aborting when A is impossible to win clearly does not affect the winning probability. We have $\varepsilon_4 = \varepsilon_3$.

G₅: Embed γ into the i_* -th H_b query. This game uses γ to answer the i_* -th distinct H_b query. As α is uniform and independent of γ , and $\mathbf{X}_* \in \text{Span}\{\mathbf{g}\}$, we know that $\mathbf{u} = \mathbf{g}^\alpha \mathbf{X}_*^\gamma$ is also independent of γ . Hence, the answer to the i_* -th H_b query is correctly distributed: uniform and independent of the previous view of A. We have $\varepsilon_5 = \varepsilon_4$.

G₆: Use α, β, γ to sign. We change SIGN_2 in this game. Recall that in previous games, SIGN_2 calls Sign_2 which sets:

$$z_1 := r_1 + bs_1 + a_1 cx_*, \quad y_1 := w_1.$$

In this game, SIGN_2 sets

$$z_1 := r_1 + bs_1 + a_1 c \cdot \frac{-\alpha - b\beta}{\gamma - b}, \quad y_1 := w_1 + a_1 c \cdot \frac{1}{\gamma - b}.$$

We remark that, since we have excluded collisions in H_b in G_1 , only the i_* -th H_b query is answered with γ . Moreover, in G_4 , we have made the game abort when A queries $(\mathcal{L}, \mu)$ to SIGN_2 if $H_b(\text{KAgg}(\mathcal{L}), \mu)$ is the i_* -th H_b query, so the game never needs to answer SIGN_2 with $H_b(\text{KAgg}(\mathcal{L}), \mu) = \gamma$. Hence, the denominator $\gamma - b$ is always non-zero.

In the following, we argue that G_6 and G_5 are identical in the view of A . We will do so by bridging these two games by a computationally unbounded simulator, which perfectly simulates both of them. Since

$$\mathbf{g}^{(-\alpha-b\beta)/(\gamma-b)}(\mathbf{uv}^b)^{1/(\gamma-b)} = \mathbf{g}^{(-\alpha-b\beta)/(\gamma-b)}(\mathbf{g}^\alpha \mathbf{X}_*^\gamma)^{1/(\gamma-b)}(\mathbf{g}^\beta \mathbf{X}_*^{-1})^{b/(\gamma-b)} = \mathbf{X}_*,$$

in G_6 , we have

$$\begin{aligned} \mathbf{g}^{z_1}(\mathbf{uv}^b)^{y_1} &= \mathbf{g}^{r_1} \mathbf{u}^{w_1} \cdot (\mathbf{g}^{s_1} \mathbf{v}^{w_1})^b \cdot (\mathbf{g}^{(-a-b\beta)/(\gamma-b)}(\mathbf{uv}^b)^{1/(\gamma-b)})^{a_1 c} \\ &= \mathbf{R}_1 \mathbf{S}_1^b \mathbf{X}_*^{a_1 c} \end{aligned}$$

as in G_5 .

We first give two intermediate simulators that simulate G_5/G_6 , respectively. Since $\mathbf{u}, \mathbf{v} \in \text{Span}\{\mathbf{g}\}$, we have $\mathbf{R}_1 = \mathbf{g}^{r_1} \mathbf{u}^{w_1}$ and $\mathbf{S}_1 = \mathbf{g}^{s_1} \mathbf{v}^{w_1}$ uniformly distributed over $\text{Span}\{\mathbf{g}\}$ and independent of w_1 . It suffices for the simulators to uniformly sample $\mathbf{R}_1, \mathbf{S}_1 \leftarrow \text{Span}\{\mathbf{g}\}$ in SIGN_1 and defer sampling $w_1 \leftarrow \mathbb{Z}_p$ to SIGN_2 . Then SIGN_2 computes the unique r_1 and s_1 such that $\mathbf{R}_1 = \mathbf{g}^{r_1} \mathbf{u}^{w_1}$ and $\mathbf{S}_1 = \mathbf{g}^{s_1} \mathbf{v}^{w_1}$, and it sets z_1 and y_1 to be as in G_5/G_6 .

Now we can give the single simulator that simulates both G_5 and G_6 . Note that for both $y_1 := w_1$ in G_5 and $y_1 := w_1 + a_1 c / (\gamma - b)$ in G_6 , uniformly sampling w_1 yields uniform y_1 . Thus, it suffices for the simulator to directly sample $y_1 \leftarrow \mathbb{Z}_p$ and compute the unique z_1 satisfying $\mathbf{g}^{z_1}(\mathbf{uv}^b)^{y_1} = \mathbf{R}_1 \mathbf{S}_1^b \mathbf{X}_*^{a_1 c}$ in SIGN_2 . Thus, the simulator perfectly simulates the views of A in both G_5 and G_6 , showing that the views are identical. We hence have $\varepsilon_6 = \varepsilon_5$.

G₇. Change $\mathbf{X}_*$. In this game, we change the distribution of $\mathbf{X}_*$. Instead of setting $\mathbf{X}_* := \mathbf{g}^{x_*}$ for uniformly random x_* , this game samples $\mathbf{X}_* \leftarrow \mathbb{G}^2$. Thanks to the change in the last game, x_* is only referenced when computing $\mathbf{X}_*$. Hence, we can reduce distinguishing between G_6 and G_7 from DDH, so $\varepsilon_7 \geq \varepsilon_6 - \varepsilon_{\text{ddh}}$.

Bound ε_7 . Finally, we bound ε_7 . We will be conditioned on $\mathbf{X}_* \notin \text{Span}\{\mathbf{g}\}$, which is true except with probability $1/p$.

We first bound the probability of a bad event: there exists $\mathcal{L}$ with $\mathbf{X}_* \in \mathcal{L}$ and $\text{KAgg}(\mathcal{L})$ having been determined such that $\text{KAgg}(\mathcal{L}) \in \text{Span}\{\mathbf{g}\}$. Recall that $\text{KAgg}(\mathcal{L}) = \prod_{i \in [\mathcal{L}]} \mathbf{X}_i^{a_i} = \mathbf{X}_*^{n_* a_1} \prod_{\mathbf{X}_i \neq \mathbf{X}_* \in \mathcal{L}} \mathbf{X}_i^{a_i}$ if $\mathbf{X}_*$ occurs n_* times in $\mathcal{L}$. Fixing the values of a_i for $\mathbf{X}_i \neq \mathbf{X}_* \in \mathcal{L}$, there exists at most one value of a_1 such that $\text{KAgg}(\mathcal{L}) \in \text{Span}\{\mathbf{g}\}$. Otherwise, if there exist $a_1 \neq a'_1$ making $\text{KAgg}(\mathcal{L}) \in \text{Span}\{\mathbf{g}\}$, then the quotient between the two $\text{KAgg}(\mathcal{L})$ values, i.e. $\mathbf{X}_*^{n_*(a_1 - a'_1)}$, must be in $\text{Span}\{\mathbf{g}\}$. This contradicts our condition $\mathbf{X}_* \notin \text{Span}\{\mathbf{g}\}$. Thus, the bad event occurs with probability at most q/p .

We now bound the probability that A wins G_7 . For A to win, it must output a valid signature σ_* on $\mathcal{L}_*$ and μ_* , where $H_b(\text{KAgg}(\mathcal{L}_*), \mu_*)$ is the i_* -th query and hence was set to γ . To be specific, A outputs a forged signature (c_*, z_*, y_*) , and the verification recovers $\mathbf{R}_*$ such that $H_c(\tilde{\mathbf{X}}_*, \mu_*, \mathbf{R}_*) = c_*$ and

$$\mathbf{R}_* \tilde{\mathbf{X}}_*^{c_*} = \mathbf{g}^{z_*} (\mathbf{uv}^\gamma)^{y_*},$$

where $\tilde{\mathbf{X}}_* = \text{KAgg}(\mathcal{L}_*)$. Recall that from G_2 , we have $\mathbf{u} := \mathbf{g}^\alpha \mathbf{X}_*^\gamma$ and $\mathbf{v} := \mathbf{g}^\beta \mathbf{X}_*^{-1}$. Therefore,

$$\mathbf{R}_* \tilde{\mathbf{X}}_*^{c_*} = \mathbf{g}^{z_*} (\mathbf{g}^\alpha \mathbf{X}_*^\gamma (\mathbf{g}^\beta \mathbf{X}_*^{-1})^\gamma)^{y_*} = \mathbf{g}^{z_* + y_* (\alpha + \beta \gamma)}.$$

This implies that $\mathbf{R}_* \tilde{\mathbf{X}}_*^{c_*} \in \text{Span}\{\mathbf{g}\}$.

Excluding the bad event, we have $\tilde{\mathbf{X}}_* \notin \text{Span}\{\mathbf{g}\}$. Consequently, $H_c(\tilde{\mathbf{X}}_*, \mu_*, \mathbf{R}_*) = c_*$ is a bad query to be defined as follows. A query $H_c(\tilde{\mathbf{X}}, \mu, \mathbf{R}) = c$ is said to be bad if $\tilde{\mathbf{X}} \notin \text{Span}\{\mathbf{g}\}$ while $\mathbf{R} \tilde{\mathbf{X}}^c \in \text{Span}\{\mathbf{g}\}$. What remains for us is to bound the probability that a bad query occurs throughout the game.

<u>Setup(1^λ)</u> 1: $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda)$ 2: $\hat{g}, u, \hat{u}, v, \hat{v} \leftarrow \mathbb{G}$ 3: $\mathbf{g} := (g, \hat{g}), \mathbf{u} := (u, \hat{u}), \mathbf{v} := (v, \hat{v})$ 4: Sample $H_b, H_c : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ 5: Sample $H_m : \{0, 1\}^* \rightarrow \mathbb{Z}_p^2$ 6: Return $(\mathbb{G}, p, \mathbf{g}, \mathbf{u}, \mathbf{v}, H_b, H_c, H_m)$ <u>KGen(n, t)</u> 7: $x \leftarrow \mathbb{Z}_p$ 8: $\{x_i\}_{i \in [n]} \leftarrow \text{Share}(x, n, t)$ 9: $\mathbf{X} := \mathbf{g}^x$ 10: for $i \neq j \in [n]$ do 11: $\text{seed}_{i,j} \leftarrow \{0, 1\}^\lambda$ 12: for $i \in [n]$ do 13: $\text{sk}_i := (x_i, \{(\text{seed}_{i,j}, \text{seed}_{j,i})\}_{j \in [n] \setminus \{i\}})$ 14: Return $\mathbf{X}, \{\text{sk}_i\}_{i \in [n]}$ <u>Sign₁(pk, k, sk_k)</u> 15: $r_k, s_k, w_k \leftarrow \mathbb{Z}_p$ 16: $\mathbf{R}_k := \mathbf{g}^{r_k} \mathbf{u}^{w_k}$ 17: $\mathbf{S}_k := \mathbf{g}^{s_k} \mathbf{v}^{w_k}$ 18: Return $(r_k, s_k, w_k), (\mathbf{R}_k, \mathbf{S}_k)$ <u>Sign₂(pk, $\mathcal{S}, \mu, k, \text{sk}_k, \text{st}_k, \mathcal{M}_1$)</u> 19: $(x_k, \{(\text{seed}_{k,i}, \text{seed}_{i,k})\}_{i \in [n] \setminus \{k\}}) := \text{sk}_k$ 20: $(r_k, s_k, w_k) := \text{st}_k$ 21: $\{\mathbf{R}_i, \mathbf{S}_i\}_{i \in \mathcal{S}} := \mathcal{M}_1$	22: $\text{sinf} := (\mathcal{S}, \mu, \mathcal{M}_1)$ 23: $b := H_b(\text{pk}, \mu)$ 24: $\mathbf{R} := \prod_{i \in \mathcal{S}} \mathbf{R}_i \mathbf{S}_i^b$ 25: $c := H_c(\text{pk}, \mu, \mathbf{R})$ 26: $z_k := r_k + bs_k + c\lambda_{\mathcal{S}, k} x_k$ 27: $y_k := w_k$ 28: $\Delta_k :=$ $\sum_{i \in \mathcal{S} \setminus \{k\}} H_m(\text{sinf}, k, i, \text{seed}_{k,i}) -$ $\sum_{i \in \mathcal{S} \setminus \{k\}} H_m(\text{sinf}, i, k, \text{seed}_{i,k})$ 29: $(\tilde{z}_k, \tilde{y}_k) := (z_k, y_k) + \Delta_k$ 30: Return $(\tilde{z}_k, \tilde{y}_k)$ <u>Combine(pk, $\mathcal{S}, \mu, \mathcal{M}_1, \mathcal{M}_2$)</u> 31: $\{\mathbf{R}_i, \mathbf{S}_i\}_{i \in \mathcal{S}} := \mathcal{M}_1$ 32: $\{(\tilde{z}_i, \tilde{y}_i)\}_{i \in \mathcal{S}} := \mathcal{M}_2$ 33: $b := H_b(\text{pk}, \mu)$ 34: $\mathbf{R} := \prod_{i \in \mathcal{S}} \mathbf{R}_i \mathbf{S}_i^b$ 35: $c := H_c(\text{pk}, \mu, \mathbf{R})$ 36: $(z, y) := \sum_{i \in \mathcal{S}} (\tilde{z}_i, \tilde{y}_i)$ 37: Return (c, z, y) <u>Vf(pk, μ, σ)</u> 38: $\mathbf{X} := \text{pk}$ 39: $(c, z, y) := \sigma$ 40: $b := H_b(\text{pk}, \mu)$ 41: $\mathbf{R} := \mathbf{g}^z (\mathbf{u} \mathbf{v}^b)^y \mathbf{X}^{-c}$ 42: Return $\llbracket c = H_c(\text{pk}, \mu, \mathbf{R}) \rrbracket$
---	--

Figure 6: HBTS2[DDH]-Mask, defined with respect to a group generation algorithm GrGen.

For each query $H_c(\tilde{\mathbf{X}}, \mu, \mathbf{R})$ with $\tilde{\mathbf{X}} \notin \text{Span}\{\mathbf{g}\}$, there exists at most one answer c that makes the query bad, i.e. $\mathbf{R}\tilde{\mathbf{X}}^c \in \text{Span}\{\mathbf{g}\}$. Otherwise, there exists another $c' \neq c$ such that $\mathbf{R}\tilde{\mathbf{X}}^{c'} \in \text{Span}\{\mathbf{g}\}$. It follows that $\tilde{\mathbf{X}}^{c-c'} \in \text{Span}\{\mathbf{g}\}$, a contradiction. Therefore, a bad query occurs throughout the game with probability at most q/p . We then conclude that $\varepsilon_7 \leq (2q+1)/p$. Putting everything together completes the proof. $\square$

5 HBTS2[DDH]-Mask: Threshold Signature Scheme with Adaptive Security

5.1 Scheme

We provide a pseudocode description of HBTS2[DDH]-Mask with respect to group generation GrGen in Fig. 6. The scheme is obtained by applying the one-time masking technique [KRT24, Che25b] to HBTS2[DDH], which is very similar to HBMS2[DDH]. In addition to sharing x , the key generation generates two symmetric keys $\text{seed}_{i,j}, \text{seed}_{j,i} \in \{0, 1\}^\lambda$ for each pair of users $i \neq j$. In the second

round of the signing protocol, each user k uses its symmetric keys to generate the mask as

$$\Delta_k := \sum_{i \in \mathcal{S} \setminus \{k\}} H_m(\text{sinf}, k, i, \text{seed}_{k,i}) - \sum_{i \in \mathcal{S} \setminus \{k\}} H_m(\text{sinf}, i, k, \text{seed}_{i,k}),$$

where sinf contains the session information $(\mathcal{S}, \mu, \mathcal{M}_1)$. Then the user masks its protocol message as $(\tilde{z}, \tilde{y}) := (z_k, y_k) + \Delta_k$.

Completeness. First, observe that the masks in a signing session are shares of zero by

$$\begin{aligned} \sum_{k \in \mathcal{S}} \Delta_k &= \sum_{k \in \mathcal{S}} \left(\sum_{i \in \mathcal{S} \setminus \{k\}} H_m(\text{sinf}, k, i, \text{seed}_{k,i}) - \sum_{i \in \mathcal{S} \setminus \{k\}} H_m(\text{sinf}, i, k, \text{seed}_{i,k}) \right) \\ &= \sum_{i \neq j \in \mathcal{S}} H_m(\text{sinf}, i, j, \text{seed}_{i,j}) - \sum_{i \neq j \in \mathcal{S}} H_m(\text{sinf}, i, j, \text{seed}_{i,j}) = 0. \end{aligned}$$

Parse $\Delta_i \in \mathbb{Z}_p^2$ as $(\Delta_{0,i}, \Delta_{1,i})$ for every $i \in \mathcal{S}$. By the following equation, we can see that the verification correctly recovers the aggregated commitment. Then the completeness follows.

$$\begin{aligned} \mathbf{g}^z (\mathbf{uv}^b)^y \mathbf{X}^{-c} &= \mathbf{g}^{\sum_{i \in \mathcal{S}} (r_i + bs_i + c\lambda_{\mathcal{S},i} x_i + \Delta_{0,i})} (\mathbf{uv}^b)^{\sum_{i \in \mathcal{S}} (w_i + \Delta_{1,i})} \mathbf{g}^{-cx} \\ &= \mathbf{g}^{\sum_{i \in \mathcal{S}} \Delta_{0,i}} (\mathbf{uv}^b)^{\sum_{i \in \mathcal{S}} \Delta_{1,i}} \prod_{i \in \mathcal{S}} (\mathbf{g}^{r_i + bs_i + c\lambda_{\mathcal{S},i} x_i} (\mathbf{uv}^b)^{w_i} \mathbf{g}^{-c\lambda_{\mathcal{S},i} x_i}) \\ &= \prod_{i \in \mathcal{S}} (\mathbf{g}^{r_i} \mathbf{u}^{w_i} \mathbf{g}^{bs_i} \mathbf{v}^{bw_i}) \\ &= \prod_{i \in \mathcal{S}} (\mathbf{R}_i \mathbf{S}_i^b) = \mathbf{R}. \end{aligned}$$

5.2 Security

Theorem 2. *If DDH is $(\tau_{\text{ddh}}, \varepsilon_{\text{ddh}})$ -hard for GrGen, then HBTS2[DDH]-Mask is $(q_s, \tau_{\text{uf}}, \varepsilon_{\text{uf}})$ -adp-TS-UF in the random oracle model against any adversary that makes at most q_h hash queries, where*

$$\varepsilon_{\text{ddh}} = \frac{\varepsilon_{\text{uf}}}{q+1} - \frac{3q+1}{p} - \frac{1}{2^\lambda},$$

$q = q_s + q_h + 1$, and $\tau_{\text{uf}} \approx \tau_{\text{ddh}}$.

The proof has two parts. The first part is proving the effect of masking. Briefly speaking, the masked signature shares only provide useful information when combined into a threshold signature under the main key. As a result, we can switch to a hybrid game that only performs signing with the main secret key, instead of the secret key shares. Hence, the secret key share is only uniformly sampled when the corresponding user is corrupted. This part follows the roadmap from [KRT24, Che25b]. The second part of this proof is very similar to the proof of Theorem 1, where we switch to a hybrid game for which we can unconditionally bound the success probability of the adversary.

Proof. Let $\mathbf{A}$ be any τ_{uf} -time adversary that makes at most q_s queries to SIGN_1 and q_h hash queries to each hash oracle. We need to prove that $\mathbf{A}$'s winning probability is at most ε_{uf} . To do so, we will build a series of hybrid games. Let $\mathbf{G}_0$ be the original $\text{adp-TS-UF}_{\text{HBTS[DDH]-Mask}}^{\mathbf{A}}$ game. Eventually, we will reach a game where we can unconditionally bound $\mathbf{A}$'s winning probability. Let ε_i denote the probability that $\mathbf{A}$ wins $\mathbf{G}_i$. Then we have $\varepsilon_0 = \varepsilon_{\text{uf}}$.

In $\mathbf{G}_0$, let $\mathbf{X}$ denote the main public key and $\{x_i\}_{i \in [n]}$ denote the secret key shares. Hash functions H_b, H_c, H_m are modeled as random oracles. Without loss of generality, we assume that $\mathbf{A}$ does not make repeated queries to a random oracle. Throughout the game, we let $\mathcal{H}$ denote the set of honest signers $[n] \setminus \mathcal{C}$.

G₁: Globalize variables in Sign₂. This game aborts if SIGN₁ ever returns repeating protocol messages. Since the first-round message $(\mathbf{R}_k, \mathbf{S}_k)$ is uniformly distributed over $\text{Span}\{\mathbf{g}\}^2$, we have $\varepsilon_1 \geq \varepsilon_0 - q_s^2/p^2$.

In this game, each signing session under user k that proceeds to the second round can be uniquely identified by sinf where $\text{sinf} = (\mathcal{S}, \mu, \mathcal{M}_1)$. This is because $\mathcal{M}_1$ contains the unique first-round protocol message $(\mathbf{R}_k, \mathbf{S}_k)$ that can be only used once in SIGN₂ under user k .

We let SIGN₂ store its local variables (in the call of Sign₂) as global variables indexed by sinf . Specifically, it stores the local variables

$$r_k, s_k, w_k, b, c, z_k, y_k, \Delta_k, \tilde{z}_k, \tilde{y}_k, \mathbf{R}_k, \mathbf{S}_k$$

as global variables

$$r_{\text{sinf},k}, s_{\text{sinf},k}, w_{\text{sinf},k}, b_{\text{sinf}}, c_{\text{sinf}}, z_{\text{sinf},k}, y_{\text{sinf},k}, \Delta_{\text{sinf},k}, \tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k}, \mathbf{R}_{\text{sinf},k}, \mathbf{S}_{\text{sinf},k}.$$

respectively, which can be referenced in the whole game including other oracle calls. Those indexed by (sinf, k) are never rewritten as sinf uniquely identifies the SIGN₂ queries under each user k . On the other hand, we note that b_{sinf} and c_{sinf} are not indexed by k . Multiple queries to SIGN₂ under different users may reference the same sinf . Nevertheless, b_{sinf} and c_{sinf} are still never written by inconsistent values, as b and c in SIGN₂ are fully determined by $\text{sinf} = (\mathcal{S}, \mu, \mathcal{M}_1)$.

G₂: Replace $H_m[\text{sinf}, i, j, \text{seed}_{i,j}]$ with $T[\text{sinf}, i, j]$. Consider H_m as a lazy sampled table. In G₁, on a query $H_m(\text{sinf}, i, j, \text{seed}_{i,j})$, the oracle returns $H_m[\text{sinf}, i, j, \text{sinf}_{i,j}]$, with each table value uniformly sampled at random when referenced for the first time.

In this game, we introduce a table T . We use items $T[\text{sinf}, i, j]$ to replace $H_m[\text{sinf}, i, j, \text{seed}_{i,j}]$. That is, on each $H_m[\text{sinf}, i, j, \text{seed}]$ query, the game returns $T[\text{sinf}, i, j]$ if $\text{seed} = \text{seed}_{i,j}$. If the item is undefined, the game uniformly samples it. If $\text{seed} \neq \text{seed}_{i,j}$, the game returns $H_m[\text{sinf}, i, j, \text{seed}]$ as in G₀.

Moreover, we let SIGN₂ directly reference $T[\text{sinf}, i, j]$ instead of internally querying $H_m(\text{sinf}, i, j, \text{seed}_{i,j})$. This changes in this game are only conceptual, so we have $\varepsilon_2 = \varepsilon_1$.

G₃: Defer generating $\text{seed}_{i,j}$. In this game, we defer generating $\text{seed}_{i,j}$ until signer i or j is corrupted. That is, when user k is corrupted, generate every $\text{seed}_{k,i}$ and $\text{seed}_{i,k}$ for $i \in [n] \setminus \{k\}$ that has not been generated.

Recall that we have made SIGN₂ directly reference $T[\text{sinf}, i, j]$ instead of querying $H_m(\text{sinf}, i, j, \text{seed}_{i,j})$. As a result, SIGN₂ does not reference any $\text{seed}_{i,j}$ any more, so deferring sampling $\text{seed}_{i,j}$ does not affect SIGN₂. The view of $\mathbf{A}$ in G₂ differs from that in G₃ only if it has queried $H_m(\text{sinf}, i, j, \text{seed})$ for some sinf and $\text{seed} = \text{seed}_{i,j}$ when both signer i and j are uncorrupted: in G₂ the oracle returns $T[\text{sinf}, i, j]$, while in G₃ it returns $H_m[\text{sinf}, i, j, \text{seed}]$ as $\text{seed}_{i,j}$ has not generated at that point. $\mathbf{A}$ can query $\text{seed}_{i,j}$ to H_m before it is generated with probability at most $q_h/2^\lambda$, so we have $\varepsilon_3 \geq \varepsilon_2 - q_h/2^\lambda$.

G₄: Pack the generation of $\Delta_{\text{sinf},k}$ and T items. Prior to this game, the mask $\Delta_{\text{sinf},k} = \sum_{i \in \mathcal{S} \setminus \{k\}} T[\text{sinf}, k, i] - \sum_{i \in \mathcal{S} \setminus \{k\}} T[\text{sinf}, i, k]$ is only generated in SIGN₂. This game additionally generates $\Delta_{\text{sinf},k}$ when $H_m(\text{sinf}, k, i, \text{seed}_{k,i})$ or $H_m(\text{sinf}, i, k, \text{seed}_{i,k})$ is queried, if user k is corrupt. That is, when answering such a query, the game does not only generate $T[\text{sinf}, k, i]$ or $T[\text{sinf}, i, k]$. Instead, it samples $T[\text{sinf}, k, j]$ and $T[\text{sinf}, j, k]$ that have not been generated for every $j \in \mathcal{S} \setminus \{k\}$ for the $\mathcal{S}$ contained in sinf , and then it computes $\Delta_{\text{sinf},k}$. This change is invisible to $\mathbf{A}$, so $\varepsilon_4 = \varepsilon_3$.

As a result of this change, $T[\text{sinf}, i, j]$ is never generated individually. Its generation must be packed with the generation of $\Delta_{\text{sinf},i}$ or $\Delta_{\text{sinf},j}$. This is because when a query in the form of $H_m(\text{sinf}, i, j, \text{seed}_{i,j})$ is made, user i or j must be corrupt, as otherwise $\text{seed}_{i,j}$ has never been generated.

G₅: Reverse generating $\Delta_{\text{sinf},k}$ and T items, and defer sampling T items. In this game, we reverse the order of generating $\Delta_{\text{sinf},k}$ and the related T item (i.e., $T[\text{sinf}, k, i]$ and $T[\text{sinf}, i, k]$ for every $i \in \mathcal{S}$, where $\mathcal{S}$ is contained in sinf) when they are generated, including in SIGN_2 and H_m . Previously, we first uniformly sample each undefined related T item then compute $\Delta_{\text{sinf},k}$. In this game, we first sample $\Delta_{\text{sinf},k}$ from its distribution, and then we sample all related T items conditioned on the value of $\Delta_{\text{sinf},k}$.

The distribution of $\Delta_{\text{sinf},k}$ is as follows.

- If for each $i \in \mathcal{S} \setminus \{k\}$, $\Delta_{\text{sinf},i}$ has been defined, then all T items related to $\Delta_{\text{sinf},k}$, i.e. $T[\text{sinf}, k, i]$, $T[\text{sinf}, i, k]$ for each $i \in \mathcal{S} \setminus \{k\}$ has been defined. Thus, $\Delta_{\text{sinf},k}$ is determined to be $-\sum_{i \in \mathcal{S} \setminus \{k\}} \Delta_{\text{sinf},i}$.
- Else, there exists some $j \in \mathcal{S} \setminus \{k\}$ such that $\Delta_{\text{sinf},j}$ is undefined. Thus, items $T[\text{sinf}, k, j]$ and $T[\text{sinf}, j, k]$ are undefined and should be uniformly generated, which is guaranteed by the “packing” in the last game. Consequently, $\Delta_{\text{sinf},k}$ is uniformly distributed.

After we reverse the order, $T[\text{sinf}, i, j]$ items are no longer referenced to generate $\Delta_{\text{sinf},i}$ and $\Delta_{\text{sinf},j}$. They are only referenced when answering $H_m(\text{sinf}, i, j, \text{seed}_{i,j})$, which only occurs after user i or j are corrupted. Thus, we can defer the sampling of the related T items until user k is corrupted. These items are sampled conditioned on the value of $\Delta_{\text{sinf},k}$. Specifically, the procedure is to uniformly sample all but one, and then the last related T is determined. The changes in this game do not affect the view of A. We have $\varepsilon_5 = \varepsilon_4$.

G₆: Alternative way to compute the last $\Delta_{\text{sinf},k}$. In this game, we change the way to compute $\Delta_{\text{sinf},k}$ for the last $k \in \mathcal{S}$, i.e. when $\Delta_{\text{sinf},i}$ has been defined for every $i \in \mathcal{S} \setminus \{k\}$. Our goal is to compute $\Delta_{\text{sinf},k}$ without referencing $\Delta_{\text{sinf},i}$ or x_i if $i \in \mathcal{H}$.

Recall that $\Delta_{\text{sinf},k} = -\sum_{i \in \mathcal{S} \setminus \{k\}} \Delta_{\text{sinf},i}$. For $i \in \mathcal{H}$, $\Delta_{\text{sinf},i}$ can only be defined in $\text{SIGN}_2(i, \text{sid}, \mathcal{S}, \mu, \mathcal{M}_1)$ for some sid and $(\mathcal{S}, \mu, \mathcal{M}_1) = \text{sinf}$. We substitute

$$\begin{aligned} \Delta_{\text{sinf},i} &= \begin{bmatrix} \tilde{z}_{\text{sinf},i} \\ \tilde{y}_{\text{sinf},i} \end{bmatrix} - \begin{bmatrix} z_{\text{sinf},i} \\ y_{\text{sinf},i} \end{bmatrix} \\ &= \begin{bmatrix} \tilde{z}_{\text{sinf},i} \\ \tilde{y}_{\text{sinf},i} \end{bmatrix} - \begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} + c_{\text{sinf}} \lambda_{\mathcal{S},i} x_i \\ w_{\text{sinf},i} \end{bmatrix} \end{aligned}$$

into $\Delta_{\text{sinf},k} = -\sum_{i \in \mathcal{S} \setminus \{k\}} \Delta_{\text{sinf},i}$. For $k \in \mathcal{H}$, we get

$$\begin{aligned} \Delta_{\text{sinf},k} &= -\sum_{i \in (\mathcal{S} \cap \mathcal{H}) \setminus \{k\}} \left(\begin{bmatrix} \tilde{z}_{\text{sinf},i} \\ \tilde{y}_{\text{sinf},i} \end{bmatrix} - \begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} + c_{\text{sinf}} \lambda_{\mathcal{S},i} x_i \\ w_{\text{sinf},i} \end{bmatrix} \right) - \sum_{i \in \mathcal{S} \cap \mathcal{C}} \Delta_{\text{sinf},i} \\ &= \sum_{i \in (\mathcal{S} \cap \mathcal{H}) \setminus \{k\}} \left(\begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} - \tilde{z}_{\text{sinf},i} \\ w_{\text{sinf},i} - \tilde{y}_{\text{sinf},i} \end{bmatrix} \right) + c_{\text{sinf}} \left[x - \sum_{i \in (\mathcal{S} \cap \mathcal{C}) \cup \{k\}} \lambda_{\mathcal{S},i} x_i \right] \\ &\quad - \sum_{i \in \mathcal{S} \cap \mathcal{C}} \Delta_{\text{sinf},i}. \end{aligned} \tag{1}$$

In the second equation, we have used the reconstruction of Shamir’s secret sharing: $x = \sum_{i \in \mathcal{S}} \lambda_{\mathcal{S},i} x_i$. Similarly, for $k \in \mathcal{C}$, we get

$$\begin{aligned} \Delta_{\text{sinf},k} &= -\sum_{i \in \mathcal{S} \cap \mathcal{H}} \left(\begin{bmatrix} \tilde{z}_{\text{sinf},i} \\ \tilde{y}_{\text{sinf},i} \end{bmatrix} - \begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} + c_{\text{sinf}} \lambda_{\mathcal{S},i} x_i \\ w_{\text{sinf},i} \end{bmatrix} \right) - \sum_{i \in (\mathcal{S} \cap \mathcal{C}) \setminus \{k\}} \Delta_{\text{sinf},i} \\ &= \sum_{i \in \mathcal{S} \cap \mathcal{H}} \left(\begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} - \tilde{z}_{\text{sinf},i} \\ w_{\text{sinf},i} - \tilde{y}_{\text{sinf},i} \end{bmatrix} \right) + c_{\text{sinf}} \left[x - \sum_{i \in \mathcal{S} \cap \mathcal{C}} \lambda_{\mathcal{S},i} x_i \right] \\ &\quad - \sum_{i \in (\mathcal{S} \cap \mathcal{C}) \setminus \{k\}} \Delta_{\text{sinf},i}. \end{aligned} \tag{2}$$

In this game, we just change to an equivalent a way to compute $\Delta_{\text{sinf},k}$ when it is deterministic. We have $\varepsilon_6 = \varepsilon_5$.

G₇. Reverse generating $(\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k})$ and $\Delta_{\text{sinf},k}$. In this game, we reverse the order of generating $(\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k})$ and $\Delta_{\text{sinf},k}$ in SIGN_2 . That is, we directly sample $(\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k})$ from its distribution, and then compute $\Delta_{\text{sinf},k} = (\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k}) - (z_{\text{sinf},k}, y_{\text{sinf},k})$.

The distribution of $(\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k})$ is the following:

- If for each $i \in \mathcal{S} \setminus \{k\}$, $\Delta_{\text{sinf},i}$ has been defined, then $\Delta_{\text{sinf},k}$ is determined and to be computed by Eq. (1). Thus, $(\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k})$ is computed by

$$\begin{aligned} \begin{bmatrix} \tilde{z}_{\text{sinf},k} \\ \tilde{y}_{\text{sinf},k} \end{bmatrix} &= \begin{bmatrix} z_{\text{sinf},k} \\ y_{\text{sinf},k} \end{bmatrix} + \Delta_{\text{sinf},k} \\ &= \begin{bmatrix} r_{\text{sinf},k} + b_{\text{sinf}} s_{\text{sinf},k} + c_{\text{sinf}} \lambda_{\mathcal{S},k} x_k \\ w_{\text{sinf},k} \end{bmatrix} + \Delta_{\text{sinf},k} \\ &= \sum_{i \in \mathcal{S} \cap \mathcal{H}} \begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} \\ w_{\text{sinf},i} \end{bmatrix} - \sum_{i \in (\mathcal{S} \cap \mathcal{H}) \setminus \{k\}} \begin{bmatrix} \tilde{z}_{\text{sinf},i} \\ \tilde{y}_{\text{sinf},i} \end{bmatrix} \\ &\quad + c_{\text{sinf}} \begin{bmatrix} x - \sum_{i \in \mathcal{S} \cap \mathcal{C}} \lambda_{\mathcal{S},i} x_i \\ 0 \end{bmatrix} - \sum_{i \in \mathcal{S} \cap \mathcal{C}} \Delta_{\text{sinf},i}. \end{aligned} \tag{3}$$

- Else, there exists some $j \in \mathcal{S} \setminus \{k\}$ such that $\Delta_{\text{sinf},j}$ is undefined. Then $\Delta_{\text{sinf},k}$ is uniformly distributed, so is $(\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k})$.

The change does not affect the view of **A**. We have $\varepsilon_7 = \varepsilon_6$.

G₈: Defer generating x_k . In this game, we defer the generation or computation of a series of variables.

- Prior to the last game, the *specific value* of $\Delta_{\text{sinf},k}$ is only referenced when computing $(\tilde{z}_{\text{sinf},k}, \tilde{y}_{\text{sinf},k})$ or when $\text{T}[\text{sinf}, k, i]$ or $\text{T}[\text{sinf}, i, k]$ needs to be defined after k is corrupted. The former occurs no more after the last game. Therefore, we can defer computing *specific value* of $\Delta_{\text{sinf},k}$ until user k is corrupted by assigning a pending symbol $\Delta_{\text{sinf},k} := ?$ for uncorrupted k .

We stress that, by assigning $?$ to $\Delta_{\text{sinf},k}$, we only defer the computation of the value of $\Delta_{\text{sinf},k}$, but not its definition. This is because the state of being defined or undefined is still referenced before k is corrupted, when deciding which way to generate $\Delta_{\text{sinf},j}$ or $(\tilde{z}_{\text{sinf},j}, \tilde{y}_{\text{sinf},j})$ for other $j \in \mathcal{S} \setminus \{k\}$.

- Next, note that in the last game, $(z_{\text{sinf},k}, y_{\text{sinf},k})$ is only referenced to compute the value of $\Delta_{\text{sinf},k}$. Since the computation has been deferred, we can also defer generating $(z_{\text{sinf},k}, y_{\text{sinf},k})$ to the corruption of k .
- Next, note that x_k is only referenced when: computing $z_{\text{sinf},k}$; being output in $\text{CORR}(k)$; computing Eqs. (2) and (3). We has just deferred the generation of $z_{\text{sinf},k}$, and Eqs. (2) and (3) only reference x_i for $i \in \mathcal{S} \cap \mathcal{C}$. Therefore, $\{x_i\}_{i \in [n]}$ do not need to be generated in the game setup. Instead, we defer sampling x_k to $\text{CORR}(k)$. Since $x, x_1, \dots, x_n$ are $(t+1)$ -wise independent, and there are at most t corruptions, x_k is always uniformly sampled when generated.

All the changes here do not affect the view of **A**. We have $\varepsilon_8 = \varepsilon_7$.

We summarize the first part of this proof by a brief takeaway. Now in G_8 , x_k is only uniformly sampled when user k is corrupted. The game never signs with the secret key shares. It only “signs” using the main secret key x , which means computing the term

$$\sum_{i \in \mathcal{S} \cap \mathcal{H}} \begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} \\ w_{\text{sinf},i} \end{bmatrix} + c_{\text{sinf}} \begin{bmatrix} x \\ 0 \end{bmatrix}$$

in Eqs. (2) and (3). Other terms in Eqs. (2) and (3) are publicly known.

The remaining part of this proof is similar to the proof of Theorem 1. We omit the details of those steps that have clear analogues in the proof of Theorem 1. The total number of queries to each hash oracle, including external queries from $\mathbf{A}$ and internal queries from other oracles and the verification of the final forgery, is bounded by q . One can verify that this bound holds for every hybrid game in the following.

G₉: Rule out collisions in H_b . The game aborts if there exist distinct H_b queries with the same returned values. We have $\varepsilon_9 \geq \varepsilon_8 - q^2/p$.

G₁₀: Generate $\mathbf{u}$ and $\mathbf{v}$ as combinations of $\mathbf{g}$ and $\mathbf{X}$. This game generates $\mathbf{u}$ and $\mathbf{v}$ to be $\mathbf{u} := \mathbf{g}^\alpha \mathbf{X}^\gamma$ and $\mathbf{v} := \mathbf{g}^\beta \mathbf{X}^{-1}$ for uniformly random $\alpha, \beta, \gamma \leftarrow \mathbb{Z}_p$. We have $\varepsilon_{10} \geq \varepsilon_9 - \varepsilon_{\text{ddh}} - 1/p$.

G₁₁: Guess the index of $H_b(\mathbf{X}, \mu_*)$. The game uniformly chooses $i_* \leftarrow [q]$ in the setup phase. For $\mathbf{A}$ to win, it is additionally required that $H_b(\mathbf{X}, \mu_*)$ is the i_* -th distinct query made to H_b . We have $\varepsilon_{11} = \varepsilon_{10}/q$.

G₁₂: Abort in advance if impossible to win. The game aborts when $\mathbf{A}$ queries μ to SIGN_2 such that $H_b(\mathbf{X}, \mu)$ is the i_* -th distinct H_b query. We have $\varepsilon_{12} = \varepsilon_{11}$.

G₁₃: Embed γ into the i_* -th H_b query. The game answers the i_* -th H_b query with γ . We have $\varepsilon_{13} = \varepsilon_{12}$.

G₁₄: Use α, β, γ to sign. Previous games need to compute the following term in Eqs. (2) and (3):

$$\sum_{i \in \mathcal{S} \cap \mathcal{H}} \begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} \\ w_{\text{sinf},i} \end{bmatrix} + c_{\text{sinf}} \begin{bmatrix} x \\ 0 \end{bmatrix},$$

where $\mathcal{S}$ is contained in sinf . We use $\begin{bmatrix} \hat{z}_{\text{sinf}} \\ \hat{y}_{\text{sinf}} \end{bmatrix}$ to denote this term. This game replaces this term in Eqs. (2) and (3) with

$$\begin{bmatrix} \hat{z}_{\text{sinf}} \\ \hat{y}_{\text{sinf}} \end{bmatrix} := \sum_{i \in \mathcal{S} \cap \mathcal{H}} \begin{bmatrix} r_{\text{sinf},i} + b_{\text{sinf}} s_{\text{sinf},i} \\ w_{\text{sinf},i} \end{bmatrix} + c_{\text{sinf}} \begin{bmatrix} -(\alpha + b_{\text{sinf}} \beta)/(\gamma - b_{\text{sinf}}) \\ 1/(\gamma - b_{\text{sinf}}) \end{bmatrix}.$$

Eq. (3) is computed in SIGN_2 . Eq. (2) is computed to generate $\Delta_{\text{sinf},k}$ when $\Delta_{\text{sinf},i}$ has been defined for every $i \in \mathcal{S} \setminus \{k\}$. At least one $i \in \mathcal{S}$ is uncorrupt, so at least one of these $\Delta_{\text{sinf},i}$ is (was) generated in SIGN_2 . G_{12} has guaranteed that the game does not need to answer SIGN_2 query that corresponds to the i_* -th H_b query. Since H_b has no collision, only the i_* -th H_b query returns γ . Therefore, we conclude that $b_{\text{sinf}} \neq \gamma$ is Eqs. (2) and (3), so the denominator $\gamma - b \neq 0$.

We argue that G_{14} is identical to G_{13} in the view of $\mathbf{A}$ by giving an (inefficient) simulator that perfectly simulates both games. It can be verified that $\mathbf{g}^{\hat{z}_{\text{sinf}}}(\mathbf{u}\mathbf{v}^{b_{\text{sinf}}})^{\hat{y}_{\text{sinf}}} = \prod_{i \in \mathcal{S} \cap \mathcal{H}} (\mathbf{R}_{\text{sinf},i} \mathbf{S}_{\text{sinf},i}^{b_{\text{sinf}}}) \mathbf{X}^{c_{\text{sinf}}}$ in both G_{13} and G_{14} .

Since $\mathbf{u}, \mathbf{v} \in \text{Span}\{\mathbf{g}\}$, SIGN_1 outputs $\mathbf{R}_k = \mathbf{g}^{r_k} \mathbf{u}^{w_k}$ and $\mathbf{S}_k = \mathbf{g}^{s_k} \mathbf{v}^{w_k}$ that are uniformly distributed over $\text{Span}\{\mathbf{g}\}$ and independent of w_k . We first consider two intermediate simulators (for G_{13} and G_{14} , respectively) that uniformly samples every $\mathbf{R}_k, \mathbf{S}_k$ in SIGN_1 and defer the generation of r_k, s_k, w_k until they are referenced. Specifically:

- When user k is corrupted, the simulators need to output r_k, s_k, w_k for each SIGN_1 query under user k . They uniformly sample $w_k \leftarrow \mathbb{Z}_p$ and computes the unique r_k, s_k satisfying $\mathbf{R}_k = \mathbf{g}^{r_k} \mathbf{u}^{w_k}$ and $\mathbf{S}_k = \mathbf{g}^{s_k} \mathbf{v}^{w_k}$.

- When the game needs to compute the term $\begin{bmatrix} \hat{z}_{\text{sinf}} \\ \hat{y}_{\text{sinf}} \end{bmatrix}$, the simulators sample $\hat{w}_{\text{sinf}} \leftarrow \mathbb{Z}_p$ and compute the unique $\hat{r}_{\text{sinf}}, \hat{s}_{\text{sinf}}$ satisfying $\prod_{i \in \mathcal{S} \cap \mathcal{H}} \mathbf{R}_{\text{sinf},i} = \mathbf{g}^{\hat{r}_{\text{sinf}}} \mathbf{u}^{\hat{w}_{\text{sinf}}}$ and $\prod_{i \in \mathcal{S} \cap \mathcal{H}} \mathbf{S}_{\text{sinf},i} = \mathbf{g}^{\hat{s}_{\text{sinf}}} \mathbf{v}^{\hat{w}_{\text{sinf}}}$. Namely, $\hat{r}_{\text{sinf}}, \hat{s}_{\text{sinf}}$, and $\hat{w}_{\text{sinf}}$ are generated as the sums $\sum_{i \in \mathcal{S} \cap \mathcal{H}} r_{\text{sinf},i}$, $\sum_{i \in \mathcal{S} \cap \mathcal{H}} s_{\text{sinf},i}$, and $\sum_{i \in \mathcal{S} \cap \mathcal{H}} w_{\text{sinf},i}$, respectively. Then the simulator for $\mathbf{G}_{13}$ computes

$$\begin{bmatrix} \hat{z}_{\text{sinf}} \\ \hat{y}_{\text{sinf}} \end{bmatrix} := \begin{bmatrix} \hat{r}_{\text{sinf}} + b_{\text{sinf}} \hat{s}_{\text{sinf}} \\ \hat{w}_{\text{sinf}} \end{bmatrix} + c_{\text{sinf}} \begin{bmatrix} x \\ 0 \end{bmatrix},$$

and the simulator for $\mathbf{G}_{14}$ computes

$$\begin{bmatrix} \hat{z}_{\text{sinf}} \\ \hat{y}_{\text{sinf}} \end{bmatrix} := \begin{bmatrix} \hat{r}_{\text{sinf}} + b_{\text{sinf}} \hat{s}_{\text{sinf}} \\ \hat{w}_{\text{sinf}} \end{bmatrix} + c_{\text{sinf}} \begin{bmatrix} -(\alpha + b_{\text{sinf}}\beta)/(\gamma - b_{\text{sinf}}) \\ 1/(\gamma - b_{\text{sinf}}) \end{bmatrix}.$$

If the simulators have sampled $\hat{w}_{\text{sinf}} \leftarrow \mathbb{Z}_p$, they should sample $w_{\text{sinf},i}$ for $i \in \mathcal{S} \cap \mathcal{H}$ (when i is corrupted) under the constraint of $\sum_{i \in \mathcal{S} \cap \mathcal{H}} w_{\text{sinf},i} = \hat{w}_{\text{sinf}}$. We note that this does not really affect the distribution of $w_{\text{sinf},i}$, as users in $\mathcal{S} \cap \mathcal{H}$ cannot be all corrupted, so the simulators can still sample each $w_{\text{sinf},i}$ uniformly from $\mathbb{Z}_p$.

Finally, we reach a single simulator that simulates both $\mathbf{G}_{13}$ and $\mathbf{G}_{14}$. Since $\hat{w}_{\text{sinf}}$ is uniformly distributed, we know that $\hat{y}_{\text{sinf}}$ is also uniformly distributed in both $\mathbf{G}_{13}$ and $\mathbf{G}_{14}$. The simulator simply samples $\hat{y}_{\text{sinf}} \leftarrow \mathbb{Z}_p$ and computes the unique $\hat{z}_{\text{sinf}}$ such that $\mathbf{g}^{\hat{z}_{\text{sinf}}} (\mathbf{u} \mathbf{v}^{b_{\text{sinf}}})^{\hat{y}_{\text{sinf}}} = \prod_{i \in \mathcal{S} \cap \mathcal{H}} (\mathbf{R}_{\text{sinf},i} \mathbf{S}_{\text{sinf},i}^{b_{\text{sinf}}}) \mathbf{X}^{c_{\text{sinf}}}$.

In conclusion, we have $\varepsilon_{14} = \varepsilon_{13}$.

$\mathbf{G}_{15}$: Change $\mathbf{X}$. This game samples $\mathbf{X}$ uniformly random from $\mathbb{G}^2$. We have $\varepsilon_{15} \geq \varepsilon_{14} - \varepsilon_{\text{ddh}}$.

Bound ε_{15} . This is the same as how we bound ε_7 in the proof of Theorem 1, except that here we do not need to bound the bad event about KAgg . We can get a bound of $\varepsilon_{15} \leq (q+1)/p$. $\square$

References

- [AB21] Handan Kilingç Alper and Jeffrey Burdges. Two-round trip Schnorr multi-signatures via delinearized witnesses. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 157–188, Virtual Event, August 2021. Springer, Cham. [2](#), [3](#), [4](#), [8](#)
- [AF04] Masayuki Abe and Serge Fehr. Adaptively secure feldman VSS and applications to universally-composable threshold cryptography. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 317–334. Springer, Berlin, Heidelberg, August 2004. [5](#)
- [AFLT16] Michel Abdalla, Pierre-Alain Fouque, Vadim Lyubashevsky, and Mehdi Tibouchi. Tightly secure signatures from lossy identification schemes. *Journal of Cryptology*, 29(3):597–631, July 2016. [8](#)
- [BB04] Dan Boneh and Xavier Boyen. Efficient selective-ID secure identity based encryption without random oracles. In Christian Cachin and Jan Camenisch, editors, *EURO-CRYPT 2004*, volume 3027 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 2004. [7](#)
- [BCJ08] Ali Bagherzandi, Jung Hee Cheon, and Stanislaw Jarecki. Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma. In Peng Ning, Paul F. Syverson, and Somesh Jha, editors, *ACM CCS 2008*, pages 449–458. ACM Press, October 2008. [4](#)

- [BCK⁺22] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022. 2, 5, 13
- [BCL⁺25] Renas Bacho, Yanbo Chen, Julian Loss, Stefano Tessaro, and Chenzhi Zhu. Adaptively secure partially non-interactive threshold schnorr signatures in the AGM. Cryptology ePrint Archive, Report 2025/1953, 2025. 2, 3, 4, 5, 9
- [BD21] Mihir Bellare and Wei Dai. Chain reductions for multi-signatures and the HBMS scheme. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 650–678. Springer, Cham, December 2021. 2, 3, 4, 5, 6
- [BDLR25a] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold schnorr signatures from DDH. Cryptology ePrint Archive, Report 2025/1009, 2025.
- [BDLR25b] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold schnorr signatures from DDH. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 390–422. Springer, Cham, August 2025. 5
- [BDLR25c] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 304–334. Springer, Cham, May 2025. 5
- [BDN18] Dan Boneh, Manu Drijvers, and Gregory Neven. Compact multi-signatures for smaller blockchains. In *International conference on the theory and application of cryptology and information security*, pages 435–464. Springer, 2018. 4
- [BDPT25] Carsten Baum, Bernardo David, Elena Pagnin, and Akira Takahashi. Universally composable interactive and ordered multi-signatures. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part II*, volume 15675 of *LNCS*, pages 3–31. Springer, Cham, May 2025. 4
- [BGC⁺25] Ruben Baecker, Paul Gerhart, Davide Li Calsi, Luigi Russo, Dominique Schröder, and Arkady Yerukhimovich. Fully adaptive FROST in the algebraic group model from falsifiable assumptions. Cryptology ePrint Archive, Report 2025/1950, 2025. 2, 4, 5, 9
- [BJ08] Ali Bagherzandi and Stanislaw Jarecki. Multisignatures using proofs of secret key possession, as secure as the Diffie-Hellman problem. In Rafail Ostrovsky, Roberto De Prisco, and Ivan Visconti, editors, *SCN 08*, volume 5229 of *LNCS*, pages 218–235. Springer, Berlin, Heidelberg, September 2008. 4
- [BL22] Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 193–207. ACM Press, November 2022. 5
- [BLL⁺21] Fabrice Benhamouda, Tancrède Lepoint, Julian Loss, Michele Orrù, and Mariana Raykova. On the (in)security of ROS. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 33–53. Springer, Cham, October 2021. 4
- [BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 514–532. Springer, Berlin, Heidelberg, December 2001. 1

- [BLSW24] Renas Bacho, Julian Loss, Gilad Stern, and Benedikt Wagner. HARTS: High-threshold, adaptively secure, and robust threshold Schnorr signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 104–140. Springer, Singapore, December 2024. 5
- [BLT⁺24] Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024. 5
- [BN06] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 390–399. ACM Press, October / November 2006. 2, 4, 10
- [Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003. 1, 4
- [BW25] Renas Bacho and Benedikt Wagner. T-spoon: Tightly secure two-round multi-signatures with key aggregation. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 256–290. Springer, Cham, August 2025. 2, 3, 4, 5, 8
- [BW26] Renas Bacho and Benedikt Wagner. Tightly secure threshold signatures over pairing-free groups. *IACR Communications in Cryptology*, 2(4), 2026. 5, 8
- [CGJ⁺99] Ran Canetti, Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Adaptive security for threshold cryptosystems. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 98–115. Springer, Berlin, Heidelberg, August 1999. 5
- [CGRS23] Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. Practical Schnorr threshold signatures without the algebraic group model. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 743–773. Springer, Cham, August 2023. 5
- [Che25a] Yanbo Chen. Dazzle: Improved adaptive threshold signatures from DDH. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part III*, volume 15676 of *LNCS*, pages 233–261. Springer, Cham, May 2025. 2, 4, 5, 8
- [Che25b] Yanbo Chen. Round-efficient adaptively secure threshold signatures with rewinding. *IACR Commun. Cryptol.*, 2(2):16, 2025. 2, 3, 4, 5, 6, 9, 18, 19
- [CKK⁺25] Elizabeth C. Crites, Jonathan Katz, Chelsea Komlo, Stefano Tessaro, and Chenzhi Zhu. On the adaptive security of FROST. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 480–511. Springer, Cham, August 2025. 2, 3, 4, 5
- [CKM23] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023. 5
- [CKM25] Elizabeth Crites, Chelsea Komlo, and Mary Maller. On the adaptive security of key-unique threshold signatures. Cryptology ePrint Archive, Report 2025/943, 2025. 2, 5

- [CS25] Elizabeth C. Crites and Alistair Stewart. A plausible attack on the adaptive security of threshold schnorr signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 457–479. Springer, Cham, August 2025. 3, 5
- [DEF⁺19] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the security of two-round multi-signatures. In *2019 IEEE Symposium on Security and Privacy*, pages 1084–1101. IEEE Computer Society Press, May 2019. 4
- [Des88] Yvo Desmedt. Society and group oriented cryptography: A new concept. In Carl Pomerance, editor, *CRYPTO’87*, volume 293 of *LNCS*, pages 120–127. Springer, Berlin, Heidelberg, August 1988. 1
- [DGNP25] Hila Dahari-Garbman, Ariel Nof, and Luke Parker. Trout: Two-round threshold ECDSA from class groups. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 380–393. ACM Press, October 2025. 5
- [FH21] Masayuki Fukumitsu and Shingo Hasegawa. A tightly secure ddh-based multisignature with public-key aggregation. *Int. J. Netw. Comput.*, 11(2):319–337, 2021. 5, 8
- [FKL18] Georg Fuchsbaauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. 2
- [FMY99] Yair Frankel, Philip MacKenzie, and Moti Yung. Adaptively-secure distributed public-key systems. In *European Symposium on Algorithms*, pages 4–27. Springer, 1999. 5
- [GCRS25] Paul Gerhart, Davide Li Calsi, Luigi Russo, and Dominique Schröder. Fully-adaptive two-round threshold schnorr signatures from DDH. Cryptology ePrint Archive, Report 2025/1478, 2025. 2, 4, 5
- [GGN16] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. Threshold-optimal DSA/ECDSA signatures and an application to Bitcoin wallet security. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider, editors, *ACNS 2016*, volume 9696 of *LNCS*, pages 156–174. Springer, Cham, June 2016. 1, 5
- [GJKR07] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, January 2007. 5
- [HDL⁺25] Calvin Abou Haidar, Dipayan Das, Anja Lehmann, Cavit Özbay, and Octavio Perez-Kempner. Privacy-preserving multi-signatures: Generic techniques and constructions without pairings. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part II*, volume 15675 of *LNCS*, pages 66–98. Springer, Cham, May 2025. 2, 3, 4
- [IN83] Kazuharu Itakura and Katsuhiko Nakamura. A public-key cryptosystem suitable for digital multisignatures. *NEC Research and Development*, 71:1–8, 1983. 1
- [JL00] Stanislaw Jarecki and Anna Lysyanskaya. Adaptively secure threshold cryptography: Introducing concurrency, removing erasures. In Bart Preneel, editor, *EURO-CRYPT 2000*, volume 1807 of *LNCS*, pages 221–242. Springer, Berlin, Heidelberg, May 2000. 5

- [KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O’Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020. 2, 3, 4, 5, 8
- [KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024. 2, 3, 5, 6, 9, 18, 19
- [KW03] Jonathan Katz and Nan Wang. Efficiency improvements for signature schemes with tight security reductions. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent Jaeger, editors, *ACM CCS 2003*, pages 155–164. ACM Press, October 2003. 8
- [Lin24] Yehuda Lindell. Simple three-round multiparty Schnorr signing with full simulatability. *CiC*, 1(1):25, 2024. 5
- [LLZD25] Yingjie Lyu, Zengpeng Li, Hong-Sheng Zhou, and Xudong Deng. Threshold ECDSA in two rounds. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 2937–2950. ACM Press, October 2025. 5
- [LÖ25] Anja Lehmann and Cavit Özbay. Putting multi into multi-signatures: Tight security for multiple signers. *Cryptology ePrint Archive*, 2025. 2, 3, 4, 5, 8
- [LOS⁺06] Steve Lu, Rafail Ostrovsky, Amit Sahai, Hovav Shacham, and Brent Waters. Sequential aggregate signatures and multisignatures without random oracles. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 465–485. Springer, Berlin, Heidelberg, May / June 2006. 4
- [LP01] Anna Lysyanskaya and Chris Peikert. Adaptive security in the threshold setting: From cryptosystems to signature schemes. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 331–350. Springer, Berlin, Heidelberg, December 2001. 5
- [Mak22] Nikolaos Makriyannis. On the classic protocol for MPC Schnorr signatures. *Cryptology ePrint Archive*, Report 2022/1332, 2022. 5
- [MOR01] Silvio Micali, Kazuo Ohta, and Leonid Reyzin. Accountable-subgroup multisignatures: Extended abstract. In Michael K. Reiter and Pierangela Samarati, editors, *ACM CCS 2001*, pages 245–254. ACM Press, November 2001. 4
- [MPSW19] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple Schnorr multi-signatures with applications to Bitcoin. *DCC*, 87(9):2139–2164, 2019. 1, 4, 6
- [MWLD10] Changshe Ma, Jian Weng, Yingjiu Li, and Robert H. Deng. Efficient discrete logarithm based multi-signature scheme in the plain public key model. *DCC*, 54(2):121–133, 2010. 4
- [NRS21] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 189–221, Virtual Event, August 2021. Springer, Cham. 2, 3, 4, 6, 7, 8
- [NRSW20] Jonas Nick, Tim Ruffing, Yannick Seurin, and Pieter Wuille. MuSig-DN: Schnorr multi-signatures with verifiably deterministic nonces. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1717–1731. ACM Press, November 2020. 3, 4

- [NRT25] Guilhem Niot, Michael Reichle, and Kaoru Takemure. Adaptively-secure three-round threshold schnorr from DL. Cryptology ePrint Archive, Report 2025/1941, 2025. [5](#)
- [Oka93] Tatsuaki Okamoto. Provably secure and practical identification schemes and corresponding signature schemes. In Ernest F. Brickell, editor, *CRYPTO'92*, volume 740 of *LNCS*, pages 31–53. Springer, Berlin, Heidelberg, August 1993. [6](#)
- [PS96] David Pointcheval and Jacques Stern. Security proofs for signature schemes. In Ueli M. Maurer, editor, *EUROCRYPT'96*, volume 1070 of *LNCS*, pages 387–398. Springer, Berlin, Heidelberg, May 1996. [2](#)
- [PW23] Jiaxin Pan and Benedikt Wagner. Chopsticks: Fork-free two-round multi-signatures from non-interactive assumptions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 597–627. Springer, Cham, April 2023. [2](#), [3](#), [4](#), [5](#), [8](#)
- [PW24] Jiaxin Pan and Benedikt Wagner. Toothpicks: More efficient fork-free two-round multi-signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 460–489. Springer, Cham, May 2024. [2](#), [3](#), [4](#), [5](#), [8](#)
- [RRJ⁺22] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. ROAST: Robust asynchronous Schnorr threshold signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2551–2564. ACM Press, November 2022. [2](#), [5](#)
- [RSY24] Lior Rotem, Gil Segev, and Eylon Yogev. From one-time to two-round reusable multi-signatures without nested forking. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part III*, volume 15366 of *LNCS*, pages 371–399. Springer, Cham, December 2024. [4](#), [5](#)
- [RY07] Thomas Ristenpart and Scott Yilek. The power of proofs-of-possession: Securing multi-party signatures against rogue-key attacks. In Moni Naor, editor, *EUROCRYPT 2007*, volume 4515 of *LNCS*, pages 228–245. Springer, Berlin, Heidelberg, May 2007. [4](#)
- [Sha79] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979. [9](#)
- [SS01] Douglas R Stinson and Reto Stroh. Provably secure distributed schnorr signatures and a (t, n) threshold scheme for implicit certificates. In *Australasian Conference on Information Security and Privacy*, pages 417–434. Springer, 2001. [5](#)
- [SS24] Gil Segev and Liat Shapira. An explicit high-moment forking lemma and its applications to the concrete security of multi-signatures. *CiC*, 1(2):2, 2024. [5](#)
- [STV⁺16] Ewa Syta, Iulia Tamas, Dylan Visser, David Isaac Wolinsky, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ismail Khoffi, and Bryan Ford. Keeping authorities “honest or bust” with decentralized witness cosigning. In *2016 IEEE Symposium on Security and Privacy*, pages 526–545. IEEE Computer Society Press, May 2016. [4](#)
- [TSS⁺24] Kaoru Takemure, Yusuke Sakai, Bagus Santoso, Goichiro Hanaoka, and Kazuo Ohta. More efficient two-round multi-signature scheme with provably secure parameters for standardized elliptic curves. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 107(7):966–988, 2024. [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#)

- [TZ23] Stefano Tessaro and Chenzhi Zhu. Threshold and multi-signature schemes from linear hash functions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 628–658. Springer, Cham, April 2023. [2](#), [3](#), [4](#), [5](#), [8](#)